

Data-driven long-term predictions of two-layer quasi-geostrophic baroclinic turbulence with deep learning models

Malene Fjeldsrud Karlsen & Olaf Wojcinski

Physics Department,
University of Oslo

GitHub page: https://github.com/Malenefk/Advanced_MachineLearning_Project/tree/main

Abstract

Predicting turbulence is of great importance for both operational use and climate research. Turbulence plays a key role in boundary layer dynamics near the land and ocean surface, and is the reason behind much of the short-term weather variability we observe. The non-linear dynamics associated with turbulence make it difficult to model accurately, and current approaches rely on computationally expensive parameterizations. This study evaluates four deep learning models: a UNet architecture, a Fourier neural Operator (FNO), and two spectral graph-based convolutional networks, to investigate whether neural networks can emulate the associated turbulence dynamics to a satisfactory extent. All four models are evaluated across five loss functions on an offline, low-resolution quasi-geostrophic dataset.

The FNO outperforms the other models with the lowest accumulated RMSE score and a physically consistent kinetic energy spectrum. The UNet retains high-frequency variance in the forecast field and is least affected by spatial smoothing. The GCN-based models are outperformed by both UNet and FNO, likely due to an overly complex graph architecture relative to the regular grid structure of the input data. Amplitude collapse is evident across all four models, with RMSE scores accumulating across forecast steps as predictions progressively lose spatial frequency and spatial structure.

This report highlights the strengths and weaknesses of different model architectures for offline turbulence emulation across five loss functions. The results suggest that model architecture has a greater influence on performance than choice of loss function, which raises questions for future work, including the use of alternative loss functions and reduced temporal sampling.

CONTENTS

I. Introduction	1
II. Theory and Method	2
A. Convolutional Neural Networks	2
B. Graph Convolutional Networks	3
C. Neural Operators	5
D. Performance Metrics	8
E. Dataset and data preparation	8
F. Prediction method	9
G. Model architectures	9
1. UNET	9
2. GCN	9
3. FNO	10
4. Simple moving average	10
H. Training loop/Hyperparameter selection	10
III. Results and discussion	12
IV. Conclusions	21
V. Appendix: Use of AI/LLM Tools	21
References	22
A. Appendix	23

I. INTRODUCTION

Ocean forecasting is of great importance in the geophysical field, especially for Norway, which stands out with the world's second-longest coastline of 24,000km (Havforskningsinstituttet, 2026). We are interested in modeling the ocean for use cases such as preparedness modeling, weather and ocean forecasting, and safe marine transportation. Modeling large-scale variables such as temperature, salinity, and the velocity field is important for forecasting the ocean state, but we also depend on modeling upper ocean turbulence.

In the upper ocean, the temporal scales are similar to those of weather forecasting. Perturbation can lead to changes in upper-ocean drift within minutes, while the deeper ocean has significantly longer temporal time scales and operates on a global scale (Berge *et al.*, 2026). The behavior of the upper ocean not only depends on waves, winds, and ocean currents, but also on turbulent mixing in a baroclinic field, where the ocean rotates under the influence of the Coriolis force in a stratified

system. These small-scale features are difficult to model due to their fine spatial and temporal scales, and they require parameterizations to represent subgrid processes in terms of larger-scale fields (Yan *et al.*, 2025), (Tziperman, 1992).

Parameterizations have several limitations: they are computationally expensive, and reduce the predictability of smaller-scale fields (Yan *et al.*, 2025). The field of Machine Learning (ML) is rapidly emerging within geophysical forecasting, where neural networks are used to predict weather and ocean states forward in time. Yan *et al.* (2025) investigated the use of Convolutional Neural Networks (CNNs) to predict geostrophic baroclinic turbulence, primarily through an online approach involving an additional fluid dynamical model used to predict a new variable S^q (Yan *et al.*, 2025).

While the results demonstrate clear benefits of using the online approach, the drawbacks include higher computational costs (Yan *et al.*, 2025). The authors mention that although including additional models yielded better results than the offline approach, many questions still remain regarding how to best tune models for this framework.

Motivated by the work of Yan *et al.* (2025), this study investigates the performance of an offline approach for forecasting two of the variables in the dataset. By leveraging CNN-based models similar to those used in the article, this study considers the architecture of a UNet model to examine how this supports forecasting of a turbulent field. CNN architectures have achieved great success in image recognition tasks, as the model assumes pixel-based inputs and leverages this structure to create more efficient forward functions (Hjorth-Jensen, 2023). CNNs are similar to ordinary neural networks, where affine transformations are applied to the input using learnable weights, and biases are added through non-linear activation functions (Hjorth-Jensen, 2023).

Recently, the use of Graph Neural Networks (GNNs) has shown competitive results in geophysical forecasting (Keisler, 2022), (Lang *et al.*, 2024), (Nipen *et al.*, 2025), (Adamov *et al.*, 2025). There are key differences between CNNs and GNNs, as GNNs operate on non-Euclidean graph structures with message-passing mechanisms, where nodes communicate information to their nearest neighbors. The key attributes of these models can be exploited in hybrid architectures such as Graph Convolutional Networks (GCNs) (Kipf and Welling, 2017). By combining message-passing graph structures with convolutional layers, GCNs provide an efficient way of encoding spatial relationships within the data.

Furthermore, a Fourier Neural Operator (FNO) is included as a spectral-domain architecture, allowing for a direct comparison of a spectral-based modeling without the graph structure found in GCNs.

The following study includes a comprehensive review of CNN and GCN architectures for predicting turbulent

variables. It is of great interest to examine how the various approaches influence forecasting performance, particularly when using physics-informed loss functions. While an ML model does not explicitly learn the underlying physical laws of geophysical systems, we can study how well the models emulate the physical behavior expected from turbulence theory, and how implementation choices and model tuning may help them reproduce these physical patterns to a greater extent.

II. THEORY AND METHOD

A. Convolutional Neural Networks

A Convolutional Neural Network is an artificial neural network designed to process and learn from 2-D grid-structured data, such as images. Learning is motivated by the use of mathematical linear operations called convolutions, which extract features from the input image using kernel filters and produce a transformed output grid. The kernel filters include learnable weights and biases that are optimized during training according to the model's task and input (Goodfellow *et al.*, 2016).

The CNNs were first introduced in 1989 by LeCun *et al.* (LeCun *et al.*, 1989) in the form of a discrete convolution for computer vision applications. The model's task was to perform simple classification on handwritten letters. The CNNs have since branched out into several directions, from object detection to image segmentation, and outperformed previous state-of-the-art methods in visual recognition (He *et al.*, 2018; Ronneberger *et al.*, 2015).

The core operations of a CNN are the discrete convolutions. For image data, such as a two-dimensional fluid dynamic snapshot denoted by $\mathbf{I}$, and a two-dimensional kernel $\mathbf{K}$. We compute the convolution by the given Equation 1 (Goodfellow *et al.*, 2016).

$$S(i, j) = (\mathbf{K} * \mathbf{I})(i, j) = \sum_m \sum_n \mathbf{I}(i - m, j - n) \mathbf{K}(m, n) \quad (1)$$

Here, an element-wise multiplication is performed between $\mathbf{I}$ and $\mathbf{K}$ while iterating through the input (i,j), with kernel (m,n), after which all products are summed together, resulting in a feature map $\mathbf{S}(i, j)$. For the feature map output, when the stride is set to 1, meaning sliding the filter by one grid pixel. The commonly applied adjustment is to extend the input dimensions by padding with values of zero around, thereby preserving the spatial dimensions of the input and output. Determining the size of the convolution output for symmetrical spatial dimensions can be simply estimated by Equation 2.

$$H_{\text{out}} = \frac{H_{\text{in}} - K_h + 2P}{S} + 1 \quad (2)$$

The formula calculates the size of the output matrix H_{out} , by the size of the input matrix H_{in} , size of the padding applied to input matrix P , the step stride S and size of the kernel K_h . Where a common approach in modern CNNs is usually to keep a symmetrical odd kernel size, either 1×1 , 3×3 , or 5×5 (Raschka et al., 2022). The content of each of the grids in the kernel is a learnable parameter called a weight, which is estimated by NN optimization. In addition, each filter also includes a bias term that is shared across the entire feature map produced by the respective kernel. The composition of the values allows the network to detect specific features from the given input data as edges or more complex features (Goodfellow et al., 2016; Raschka et al., 2022). To enable the model to learn more complex representations from the input data, CNN models use activation functions to introduce non-linearity (Raschka et al., 2022). The most popular activation function used in modern CNN's is Rectified Linear Unit (ReLU), as shown in the Equation 3.

$$A(i, j) = \max(0, S(i, j)) \quad (3)$$

The ReLU activation function applies a transformation to the input $S(i, j)$, mapping negative values to zero, while leaving non-negative values unchanged as the $A(i, j)$. The popularity of ReLU is not only due to faster training, but also to its ability to mitigate vanishing gradients that occurs when using the traditional sigmoidal activation functions (Raschka et al., 2022).

After feature extraction is performed with a kernel and an activation function applied, the created feature maps are often subsampled by a pooling layer. These pooling layers aim to introduce a degree of local translation invariance, meaning that small shifts in the spatial position of the features do not significantly affect their detection by the network (Goodfellow et al., 2016; Raschka et al., 2022). The reduction of spatial dimensions in feature maps also benefits the model in computational efficiency and mitigates overfitting by reducing the information passed to the subsequent layers (Raschka et al., 2022). The pooling layer behaves similarly to convolution as it uses a sliding window and stride, however the pooling operation does not contain any learnable weights or bias parameters. The two common forms of pooling are: Max Pooling, where the grid of maximum value within a defined neighborhood is selected, or average pooling, where the average value is computed within a given neighborhood.

Further implementation is given in Section II.G.1

B. Graph Convolutional Networks

The CNN architecture is well-suited for Euclidean data structures such as images, where it exploits hierarchical

patterns in the pixel grid (Zhang et al., 2019). However, when working with graph structures with a non-Euclidean geometry, traditional CNN operations are not directly applicable. To address this, Graph Neural Networks (GNNs) have been developed in Machine Learning to process irregular structures by representing data as nodes connected through edges. Through this representation, each node contains information about itself while also maintaining relationships with neighboring nodes through the connecting edges. Inspired by the convolutional operations in CNNs, where information is propagated through kernels over neighboring pixels, Graph Convolutional Networks (GCN) extend this idea to graphs (Zhang et al., 2019). A GCN aggregates information from neighboring nodes through the graph edges and processes the aggregated information using non-linear activation functions. GCN architectures are either defined in the vertex domain or in the spectral domain based on a graph Fourier transformation, which represents information as graph signals. In this work, we focus on the latter approach.

To understand how information is processed and shared between neighboring nodes in a GCN, we can consider the one-dimensional algorithm developed by Weisfeiler and Lehmann in 1968 (Kipf and Welling, 2017). This algorithm describes the propagation rule for updating a single node and is shown in equation 4.

$$h_i^{(l+1)} = \sigma(h_i^{(l)} W_0^{(l)} + \sum_{j \in \mathcal{N}_i} \frac{1}{c_{ij}} h_j^{(l)} W_1^{(l)}) \quad (4)$$

To compute the next layer representation $h_i^{(l+1)}$, information from the node itself at layer l is combined with a weighted sum of the representations of its neighboring nodes. The normalization term $c_{ij} = \sqrt{d_i d_j}$ accounts for the degree of the nodes and scales the contributions accordingly (Kipf and Welling, 2017). Here $W^{(l)}$ denotes the weight matrix at layer l , and σ represents the non-linear activation function applied to the aggregated information. In practice, this means that each node aggregates information from its neighbors before the result is passed through the neural network layer. The aggregated representation is multiplied by the weight matrix and transformed by the activation function to produce the updated node representation. To implement this propagation rule, the graph structure must be mathematically represented by adjacency and degree matrices.

The methodology used in this study is inspired by the work of Thomas N. Kipf and Max Welling (Kipf and Welling, 2017). However, because the data set used in this study differs from theirs, certain modifications are necessary in the methodology. These adjustments will be described further in the methods section.

A graph can be defined as $f(X, A)$, where X represents the input data as nodes and A represents the edges con-

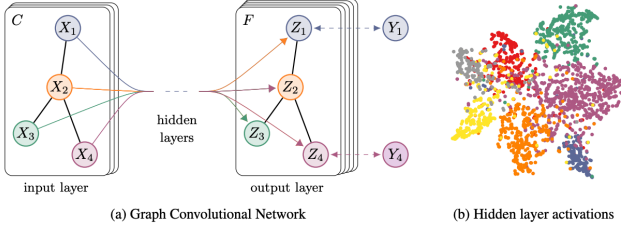


FIG. 1: Schematic view of the spectral GCN architecture. The Figure illustrates the message-passing between nodes and the corresponding activation functions. The schematic Figure is gathered from (Kipf and Welling, 2017).

necting the nodes (Defferrard and Vandergheynst, 2016). To indicate which nodes are connected, the graph structure is described using an adjacency matrix A of shape $N \times N$, containing elements a_{ij} that indicate the edges of the graph. In the matrix, the rows correspond to the source nodes and the columns to the target nodes, which describe the relationships between connected nodes. By multiplying the matrix A with a weight $W \in \mathcal{R}^{n \times n}$, an updated weighted adjacency matrix can be obtained, describing the weighted links in the graph. The weights can be defined as binary values, where 1 indicates a connection and 0 indicates no connection between nodes. However, the graph can also be weighted using real values $\mathbb{R}$, allowing the strength of the connections between nodes to vary (Hammond *et al.*, 2009).

Using the adjacency matrix, the degree of each node in the graph can be defined. The degree represents the total weight of the edges connected to a given node a_i and is represented in a diagonal matrix Δ , where the diagonal elements are given by $d_i = \sum_{j=1}^N a_{ij}$ (Zhang *et al.*, 2019). This diagonal matrix is used to construct the graph Laplacian. The standard graph Laplacian is defined as $L = D - A$, where $D = \Delta$ denotes the degree matrix and A is the adjacency matrix. The Laplacian matrix is important as it describes how signals vary across neighboring nodes and captures the connectivity in the graph structure (Sofeikov, 2025). However, because some nodes may have significantly more connections than others, as illustrated in the node connections in Figure 1, the Laplacian is normalized using Equation 5. This normalization accounts for differences in node degree and prevents nodes with a large number of connections from dominating the propagation of information in the graph (Zhang *et al.*, 2019).

$$L = I_N - D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \quad (5)$$

Using the normalized Laplacian L , we can now define the spectral convolution used in the GNN. Spectral graph convolution is performed by filtering a graph sig-

nal $x \in \mathcal{R}^N$ with a spectral filter $g_\theta(\Lambda) = \text{diag}(\theta)$ (Kipf and Welling, 2017).

$$g_\theta * x = U g_\theta(\Lambda) U^T x \quad (6)$$

Here, the element x_i represents the signal associated with a single node i in the graph. To express the signal in the spectral domain, the graph Fourier transform is introduced. The signal x can be rewritten as $\hat{x} = U^T x$, where U is the matrix of eigenvectors of L , often referred to as the graph Fourier modes (Defferrard and Vandergheynst, 2016). These eigenvectors form the Fourier basis of the graph, allowing the signal to be represented in the frequency domain rather than the vertex domain. Because the normalized Laplacian in equation 5 is symmetric, it can be decomposed as

$$L = U \Lambda U^T$$

, where $\Lambda = \text{diag}(\lambda)$ contains the eigenvalues of L , which represent the frequencies of the graph (Defferrard and Vandergheynst, 2016). Using this decomposition, the multiplication of the graph signal and the filter can be expressed as shown in equation 6, where the filtering becomes a function of the graph frequencies $g_\theta(\Lambda)$ (Kipf and Welling, 2017).

However, computing the eigendecomposition of L is computationally expensive for large graphs. The filter g_θ is parameterized by $\theta \in \mathcal{R}^N$, where θ is the parameter that determines how graph frequencies are weighted during filtering (Defferrard and Vandergheynst, 2016). However, since g_θ is a non-parametric filter in which all parameters are free, this formulation introduces several limitations when filtering the graph signals (Defferrard and Vandergheynst, 2016). First, the filter is not localized in the vertex domain, meaning the filtering may influence nodes across the whole graph. Second, the learning complexity is high, as computing the eigenvector decomposition of the Laplacian is computationally expensive (Zhang *et al.*, 2019).

A common solution is to parameterize the filter using a polynomial of the Laplacian. By doing so, the filter becomes localized to the graph structure and becomes "K-localized", meaning the filter only affects nodes within K steps of a given node, often referred to as the K -neighborhood (Kipf and Welling, 2017). This approach is useful as it allows the creator of the architecture to determine how far information propagates in the graph, while significantly reducing the computational cost of the convolution.

To further improve computational efficiency, Chebyshev polynomials are introduced to approximate the filter g_θ . The Chebyshev polynomials are defined as shown in equation 7, where $T_0(x) = 1$ and $T_1(x) = x$ (Kipf and Welling, 2017).

$$T_K(x) = 2xT_{K-1}(x) - T_{K-2}(x) \quad (7)$$

Using these polynomials, the spectral filter can be approximated as a truncated expansion, as shown in equation 8 (Kipf and Welling, 2017).

$$g'_\theta(\Lambda) \approx \sum_{k=0}^K \theta'_k T_k(\hat{\Lambda}) \quad (8)$$

This approximation allows the convolution operation to be calculated without needing to explicitly compute the eigenvector decomposition. The convolution of the graph signal can now be written as

$$g'_\theta * x \approx \sum_{k=0}^K \theta'_k T_k(\tilde{L})x \quad (9)$$

where the scaled Laplacian is defined as $\tilde{L} = \frac{2}{\lambda_{max}}L - I_N$ (Kipf and Welling, 2017).

By expressing the filter as a polynomial, the operation becomes K -localized. Additionally, the computational complexity is significantly reduced, as the convolutions are computed with a complexity proportional to the number of edges in the graph, ie, linear to the number of edges.

In the Kipf and Welling (2017) paper, the largest eigenvalue λ_{max} is set to a fixed scalar value of 2 to simplify equation 9. The authors argue that the neural network parameters will adapt to the scaling during training (Kipf and Welling, 2017). Although the normalized Laplacian is often approximated by using a fixed scalar, we have chosen to directly estimate the value of λ_{max} in order to avoid numerical instability and to ensure the spectral scaling is consistent with the data. Following an approach proposed in a related paper, the value of λ_{max} is obtained by computing the Arnoldi iteration or the Jacobi-Davidson method, where $\lambda_{max} \geq \lambda_{N-1}$ (Defferrard and Vandergheynst, 2016).

To further simplify the model and to obtain a linear function with respect to the Laplacian, we set $K = 1$. This is the same as performing a first-order approximation of the Chebyshev polynomial and allows multiple convolutional layers to be stacked when building the model, as proposed in (Kipf and Welling, 2017). This is efficient for building deeper models while reducing the risk of overfitting in the neighborhood aggregation. With this simplification, equation 9 reduces to equation 11. The parameter λ_{max} is used to obtain a general representation of the spectral convolution, while the estimation of the scalar value is introduced later in section II.G.2.

$$\begin{aligned} g'_\theta * x &\approx \theta'_0 x + \theta'_1 \left(\frac{2}{\lambda_{max}} L - I_N \right) x \\ &= \theta'_0 x + \theta'_1 \left(\frac{2}{\lambda_{max}} D^{-\frac{1}{2}} A D^{-\frac{1}{2}} + \frac{2}{\lambda_{max}} I_N - I_N \right) x \end{aligned} \quad (10)$$

$$(11)$$

The theory presented above is the mathematical foundation of GNNs and the model used in the project. The model architecture and implementation are described in subsection II.G.2.

C. Neural Operators

The motivation for using neural operators lies in their ability to address some fundamental limitations in data-driven physics research. Whereas standard NN- and CNN- based models typically learn mappings between vectors in finite-dimensional Euclidean spaces (Li *et al.*, 2021), which may be unsuitable for specific scientific data modeling tasks. Neural operators are advantageous because they are designed to be discretization-invariant. In this context, discretization-invariant means that the model is not fixed to a given grid resolution. The same parameters can be used on different grid resolutions. This is achieved by learning an approximation of the underlying operator $\mathcal{G}$ itself. This allows the model to learn the mapping between an input function $a \in \mathcal{A}$ and an output solution function $u \in \mathcal{U}$. As given in the Equation 12, the $\mathcal{A}$ and $\mathcal{U}$ denote two separate Banach spaces (Azizzadenesheli *et al.*, 2024; Li *et al.*, 2021). Thus, the neural operator learns how changes in the input function determine the resulting output function u expressed as $\mathcal{G}(a) = u$.

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U} \quad (12)$$

In addition to discretization-invariance, the neural operators framework has proven the universal approximation theorem of continuous nonlinear operators between Banach spaces. In contrast, regular neural networks keep their universal approximation limited to functions only (Hornik *et al.*, 1989). This makes neural operators currently the only known class of model that can be discretization-invariant while approximating any continuous operator to its desired level of accuracy in suitable conditions (Kovachki *et al.*, 2023). This makes the framework powerful for physics-related scientific modeling.

Learning an operator gives the model the capability to learn the mapping between infinite-dimensional or different finite-dimensional function spaces, allowing it to represent solutions over continuous domains (Chen and Chen, 1993, 1995). Rather than only learning specific patterns for predicting solutions, neural operators learn the underlying family of partial differential equations (Li *et al.*, 2021). This is an important advantage

compared with traditional NN and CNN models, which are resolution-bounded. Such models require the use of additional interpolation techniques, such as bilinear or trilinear interpolation, to make such predictions in continuous domains. This disadvantage results in additional complexity and often poor performance, due to their limited ability to capture finer details (Azizzadenesheli et al., 2024). Neural operator, therefore, greatly benefits prediction tasks such as fluid dynamics, weather forecasting, or heat propagation, and can make predictions at an arbitrary point within the continuous spatial domains. An additional benefit of neural operators is their ability to perform so-called *zero-shot super-resolution*, which means that they can generalize and effectively predict a higher resolution output, such as 128×128 or 256×256 grids, even if the model has been trained exclusively on 64×64 data grid (Kovachki et al., 2023).

The first practical application and introduction of the neural operator framework, inspired by research on continuous neural networks, was presented by Li et al. (2020). *Graph Kernel Network for partial Differential Equations* (Li et al., 2020), following earlier work on learning nonlinear operators such as DeepONet (Lu et al., 2021). The paper aimed to learn mappings between infinite-dimensional spaces, where the benchmark was a synthetic elliptic PDE problem commonly used in hydrology and elasticity. Their first model treated the physical domain as a graph, where discretized spatial locations were represented as nodes, and their interactions were edges. The model further used a message passing algorithm to approximate its kernel, allowing the model to be compatible with unstructured grids (Li et al., 2020). Despite the model’s flexibility and performance, it suffered from high computational complexity and quadratically scaling costs with the number of nodes due to the capture of pairwise information between every two nodes in the graph (Li et al., 2020). The quadratically scaling in the graph kernel network (GKN) is given by Equation 13, where N is the number of nodes, indicating grid points.

$$\mathcal{O}(N^2) \quad (13)$$

Fourier Neural Operators (FNO) were first introduced one year later in the research paper by Li et al. (2021) *Fourier Neural Operator for Parametric Partial Differential Equations* (Li et al., 2021). The FNO model replaced the computationally expensive graph structure and integral kernel in physical space with the spectral Fourier domain, addressing the model’s bottleneck. This allowed the model to replace expensive integral kernel approximations, using convolutions, by utilizing the fast Fourier transform (FFT). These operations became simple element-wise multiplications, resulting in quasi-linear complexity growth as given in the Equation 14 (Azizzadenesheli et al., 2024), where N is the number of input grid points. This is achieved while preserving the neural operator’s universal approximation capabilities and

discretization-invariance (Kovachki et al., 2021; Li et al., 2021)

$$\mathcal{O}(N \log(N)) \quad (14)$$

The success of the FNO is also attributed to the ability to capture global dependencies and long-range interactions within a physical domain. This is achieved by performing efficient computations simultaneously across the entire Fourier space, allowing the FNO model to learn how different frequency modes contribute to the underlying physical system (Goswami et al., 2022). Consequently, the FNO is better suited to learning non-local interactions than its precursor GKN, which is more constrained by local receptive fields (Azizzadenesheli et al., 2024; Li et al., 2021). Applications of FNO include modern machine learning research on turbulent flows, being the first model to successfully approximate solutions to the turbulent flows in Navier-Stokes equations with high fidelity, three to five orders of magnitude faster than traditional numerical solvers (Li et al., 2021). The model has also achieved strong results in other physical domains, such as weather forecasting (Pathak et al., 2022) and predictions of geological CO_2 reservoir storage (Wen et al., 2023).

Neural operators follow a general training formulation for learning an operator from data. Although they are formulated on continuous function spaces, they are trained on a finite set of N input-output pairs $\{(a_j, u_j)\}_{j=1}^N$ from a domain $\mathcal{D}$. Each pair consists of an input function and its corresponding output function. Since the true operator $\mathcal{G}$ is unknown, the model learns a parametric approximation $\mathcal{G} : \mathcal{A} \times \Theta \rightarrow \mathcal{U}$, or equivalent $\mathcal{G}_\theta : \mathcal{A} \rightarrow \mathcal{U}$ for fixed parameters space $\theta \in \Theta$. The aim is for θ to approximate the true operator $\mathcal{G}$ (Li et al., 2020). The design of neural operators mimics the conventional neural networks, with a key difference where layers replace finite-dimensional matrix multiplications with integral operators in function spaces (Azizzadenesheli et al., 2024; Kovachki et al., 2023). The typical architecture for neural networks consists of a three-step process where the input function $a \in \mathcal{A}$ follows a: dimensional uplifting, iteratively computing kernel integration, and lastly down projection, as illustrated in Figure 10. The model first takes an input function $a(x)$ that is defined on a domain $\mathcal{D}$ and performs a pointwise lifting into a higher-dimensional hidden representation. At each spatial location $x \in \mathcal{D}$, the input value $a(x)$ belongs to $\mathbb{R}^{d_a}$, where d_a is the number of input channels derived from the used data features. The lifting transformation maps this input to a hidden representation $v_0(x) \in \mathbb{R}^{d_{v_0}}$, where $d_{v_0} > d_a$. This transformation is expressed by Equation 15. The operator P is a parametrized pointwise neural network, implemented using a fully connected layer with a 1×1 convolution. The lifting operation embeds the original input into a richer latent feature space, allowing the model to learn more expressive representations (Kovachki et al.,

2023).

$$v_0(x) = P(a(x), x) \quad (15) \quad \mathcal{G}_\theta := Q \circ \sigma_T (W_{T-1} + K_{T-1} + b_{T-1}) \circ \cdots \circ \sigma_1 (W_0 + K_0 + b_0) \circ P \quad (20)$$

The lifted representation is then passed through T hidden layers. For $t = 0, \dots, T-1$ each layer maps its hidden representation v_t to the next hidden state v_{t+1} as shown in Equation 16. Each transformation layer is a non-linear operator that combines both local and non-local information, as shown in the Equation 17. Here W_t denotes the local linear transformation, that consist of a standard pointwise matrix multiplication on $v_t(x)$. This acts similarly to a skip connection or residual connection, regularizing the iterative updates (Kovachki *et al.*, 2023) shown at the bottom of the Figure 10. The non-local integral kernel operator $(\mathcal{K}_t v_t)(x)$ is the neural operator’s central component. This learnable kernel determines how information from a given point y influences the updated points at x . This core function allows the model to be capable of capturing long-range dependencies and interactions across the current domain (Kovachki *et al.*, 2023). The Kernel operator is given in Equations 18, where the information from points y in the current domain is summed by an integral to update the hidden representation at x . The integration is performed with respect to $d\nu(y)$. When ν is chosen as Lebesgue measure, the integral corresponds to an ordinary spatial integration by $d\nu(y) = dy$ (Kovachki *et al.*, 2023). Lastly, the term $b_t(x)$ denotes a bias function, often parametrized as a fixed vector across the whole domain. The σ denotes a non-linear activation that is applied at each location x within the domain.

$$v_0(x) \rightarrow v_1(x) \rightarrow v_2(x) \rightarrow \cdots \rightarrow v_T(x). \quad (16)$$

$$v_{t+1}(x) = \sigma(W_t v_t(x) + (\mathcal{K}_t v_t)(x) + b_t(x)), \quad t = 0, \dots, T-1. \quad (17)$$

$$(\mathcal{K}_t v_t)(x) = \int_{\mathcal{D}} \kappa_t(x, y, a(x), a(y)) v_t(y) d\nu(y). \quad (18)$$

In the final step after, T -th hidden layer $v_T(x)$, the latent space is projected back to the target function space $\mathcal{U}$, by decoding the high-dimensional feature representation into the native dimension. The transformation is performed by the pointwise neural network operator Q , as shown in Equation 19.

$$u(x) = Q(v_T(x)) \quad (19)$$

The full composition of the operators in the model’s architecture can be shown by the Equation 20, where $\mathcal{G}_\theta$ denotes the learned neural operator and θ denotes its learnable parameters.

The general neural operator architecture described above provides the foundation for FNO. The uplifting operator P , the projection operator Q , and the layered structure remain the same as given in the Equation 15, Equation 19, and Equation 20. However, the FNO architecture diverges inside the hidden layers. In both cases, the model representations are updated through T hidden layers as given in Equation 16, combining a local linear transformation W , used as a skip connection, and a non-local kernel operator $\mathcal{K}$ (Kovachki *et al.*, 2023).

The FNO optimizes the computationally expensive kernel operator $\mathcal{K}$ in Equation 18 by restricting the integral operator in each hidden layer to a convolution multiplication in the Fourier domain. The expensive integration kernel is, by the convolution theorem, transformed into a mode-wise multiplication in the spectral domain.

First, each hidden representation $v_t(x)$ is transformed from physical space into Fourier space by FFT, denoted by $\mathcal{F}$. Here k denotes the frequency mode. This is shown in Equation 21.

$$\hat{v}_t(k) = \mathcal{F}(v_t)(k) \quad (21)$$

Second, the model performs a linear transformation in Fourier space by multiplying the Fourier coefficients from a learnable weight tensor R . The FNO keeps only the fixed number of Fourier modes, controlled by the hyperparameter $K_{\max}$. This truncation removes higher-frequency modes, reducing computational cost and noise while regularizes the model, and keeping a clear signal and global dependencies from the lower frequency modes (Li *et al.*, 2021).

Finally, the Inverse Fourier Transform (IFFT) is applied to transform frequency modes back to the physical domain. The Fourier kernel operator can therefore be written as in Equation 22

$$(\mathcal{K}_t^F v_t)(x) = \mathcal{F}^{-1}(R_t \cdot \mathcal{F}(v_t))(x) \quad (22)$$

The output from the Fourier operator is then used inside the FNO layer together with the local linear transformation matrix W_t and a non-linear activation function as given in the Equation 23, similar to the general neural operator Equation 20.

$$\mathcal{G}_\theta := Q \circ \sigma_T (W_{T-1} + \mathcal{K}_{T-1}^F + b_{T-1}) \circ \cdots \circ \sigma_1 (W_0 + \mathcal{K}_0^F + b_0) \circ P \quad (23)$$

The theory presented above is the mathematical foundation of FNOs and the model used in the project. The model architecture and implementation are described in II.G.3.

D. Performance Metrics

For the model evaluation on our test set, we use four different metrics to quantify the predictive performance of the model, where the fourth metric tracks amplitude collapse across increasing timesteps using the standard deviation ratio.

Root Mean Squared Error (RMSE) is a widely used quantitative measure for assessing a model's performance in statistical regression tasks with continuous target variables. It is derived from the Mean Squared Error (MSE), but unlike MSE, RMSE is more intuitive and interpretable due to its unit of measure transformed back to the original target variable. The mathematical definition of RMSE is expressed in Equation 24. The computation consists of the square root of the average squared difference between the model prediction $\hat{Y}_{m,v,\ell,h,w}$ and the corresponding ground truth $Y_{m,v,\ell,h,w}$. The squared error ensures positive and negative errors do not cancel out, while penalizing larger errors more.

$$\text{RMSE} = \sqrt{\frac{1}{MVLHW} \sum_{\substack{m=1,\dots,M \\ v=1,\dots,V \\ \ell=1,\dots,L \\ h=1,\dots,H \\ w=1,\dots,W}} (\hat{Y}_{m,v,\ell,h,w} - Y_{m,v,\ell,h,w})^2} \quad (24)$$

Here, m indexes the test samples, v the variable or variable layer, ℓ the time steps, h and w the spatial grid locations by height and width, respectively. The total number of each index is, however, denoted by M , V , L , H , and W , respectively.

The second metric we use is the Mean Absolute Error (MAE), given by Equation 25. This is used to measure the average absolute difference between the predictions and the ground truth.

$$\text{MAE} = \frac{1}{MVLHW} \sum_{\substack{m=1,\dots,M \\ v=1,\dots,V \\ \ell=1,\dots,L \\ h=1,\dots,H \\ w=1,\dots,W}} |\hat{Y}_{m,v,\ell,h,w} - Y_{m,v,\ell,h,w}| \quad (25)$$

The third metric is the Mean Absolute Scaled Error (MASE) given by the Equation 26. MASE compares the model's MAE against the MAE of a simple naive forecast model that predicts the next timestep to be the same as the previous.

$$\text{MASE} = \frac{\frac{1}{MVLHW} \sum_{\substack{m=1,\dots,M \\ v=1,\dots,V \\ \ell=1,\dots,L \\ h=1,\dots,H \\ w=1,\dots,W}} |\hat{Y}_{m,v,\ell,h,w} - Y_{m,v,\ell,h,w}|}{\frac{1}{M(VL-1)HW} \sum_{\substack{m=1,\dots,M \\ \tau=2,\dots,VL \\ h=1,\dots,H \\ w=1,\dots,W}} |Y_{m,\tau,h,w}^{\text{flat}} - Y_{m,\tau-1,h,w}^{\text{flat}}|} \quad (26)$$

Here Y^{flat} denotes the ground truth after combining variable index v with the timestep index ℓ into one long sequence denoted as τ . This allows the denominator to measure the average of the absolute change, giving the error of the simple naive forecast model.

Finally, we use the standard deviation ratio to monitor the model's progressive amplitude behavior when predicting future timesteps. This is motivated by the tendency of neural networks to gradually predict towards the mean when uncertain. The metric is computed as standard deviation $\sigma(\cdot)$ of the predicted fields given $\hat{Y}_{:,v,\ell,:}$ divided by the corresponding standard deviation of ground truth fields $Y_{:,v,\ell,:}$ for each variable v for each timestep ℓ as shown in Equation 27. Values close to 1 indicate a preserved amplitude, with similar variability, whereas values close to 0 indicate a strong amplitude collapse, when predictions become homogeneous with little variability.

$$\text{StdRatio}_{v,\ell} = \frac{\sigma(\hat{Y}_{:,v,\ell,:})}{\sigma(Y_{:,v,\ell,:})} \quad (27)$$

E. Dataset and data preparation

To create a data-driven model aimed at predicting future states of ocean turbulence, the neural networks were trained on simulation data from a two-layer quasi-geostrophic model previously used by Yan et al. (Yan, 2025). The two-layer QG model used is a simplified simulation of plausible ocean turbulence dynamics.

The dataset consists of four, two-dimensional 64×64 grids. These correspond to each of the two physical variables given in Table I: Potential Vorticity and Stream functions, which each have two distinct vertical layers. The simulations were run on the same fixed model parameters as shown in Table V, however each of the 300 simulations was generated from a randomized set of initial conditions (Yan, 2025).

The data from Yan et al. (Yan, 2025), were processed by spectral truncation and resampling the grid from originally 256×256 down to 64×64 . To reduce temporal correlation between samples, only every 1000th timestep was retained. This changed the sampling interval from hourly snapshots to snapshots every ≈ 42 days. This preserves chronological order while ensuring that samples

are widely separated in time and less dependent on each other. In total, each simulation represents 10 years model simulation with 86 time steps over a spatial domain of 1000×1000 km.

For this project, an additional subsampling of the data has been performed. To remove the so-called spin-up phase of the QG model. The spin-up process spans approximately the first 30 timesteps and is characterized by slow, low-amplitude, repetitive motion that is not representative of a developed steady turbulent flow. The resulting dataset, therefore, excluded the first 30 timesteps in each of the 300 simulation runs, leaving each simulation with only its final 56 timesteps. The resulting distribution before and after subsampling is compared in Figure 11.

The dataset was split using a standard 70/15/15 ratio for training, validation, and test samples, respectively. The total number of time steps in each split was 47040/10080/10080, respectively. Each variable in the dataset was normalized individually using z-score normalization given in Equation 28, with the mean μ and standard deviation σ computed exclusively from the training set, separately for inputs x and targets y . These parameters were then applied to normalize the validation and test sets. This procedure was used to reduce the magnitude imbalance, obtain a more equal variable contribution, and ensure no data leakage.

$$x' = \frac{x - \mu}{\sigma} \quad (28)$$

TABLE I: Overview of the variables used in the dataset with their physical units, value ranges after cutting off spin-up

Variable	Layer	Unit	Range [Min, Max]
Stream function (ψ)	1	$\text{m}^2 \text{s}^{-1}$	$[-9.22 \times 10^3, 8.91 \times 10^3]$
Stream function (ψ)	2	$\text{m}^2 \text{s}^{-1}$	$[-2.35 \times 10^3, 2.37 \times 10^3]$
Potential vorticity (q)	1	s^{-1}	$[-5.71 \times 10^{-5}, 5.96 \times 10^{-5}]$
Potential vorticity (q)	2	s^{-1}	$[-6.40 \times 10^{-6}, 6.40 \times 10^{-6}]$

F. Prediction method

The prediction method includes forecasting time steps ahead, based on four previous timesteps for each of the variables and variable layer v . Each timestep represents approximately 42 days, setting the prediction horizon $L_{\text{out}} = 4$, the model predicts a total period of 168 days or approximately 5.6 months into the future.

The prediction method uses an iterative autoregressive rollout. This means the model predicts only one timestep at a time, starting from its given initial input window. After predicting one timestep $\ell + 1$ the oldest timestep from the input window is dropped, while the new prediction gets appended as the most recent timestep in the

new input window. This updated input window is then used to predict timestep $\ell + 2$, and this rollout mechanism is repeated until the length given by L_{out} is reached.

G. Model architectures

1. UNET

The U-Net (Ronneberger *et al.*, 2015) is a specialized type of a convolutional neural network. The model used in this project follows a similar architecture to the original U-Net architecture with an initial double convolution block, four encoder blocks, four corresponding decoder blocks, and one output convolution layer. Additionally, the model includes the two custom components: temporal positional encoding and residual prediction. The architecture starts with a double 2D convolution block with 3×3 kernels, used to extract features from the input, with padding used to preserve the spatial resolution. Each of the convolutions is followed by batch normalization and a non-linear activation function, ReLU. This block maps the input data to 64 feature channels, before the first encoder.

The first encoder module starts with a downsampling by a 2×2 max pooling operation, which progressively reduces the spatial resolution by a factor of two, while doubling the feature channels. The feature maps are then fed through the double 2D convolutional block to extract new features from the data. The forward pass performs this encoding four times from $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$.

After the fourth encoder, the first decoder starts with a 2D transposed convolution on the 1024 feature channels, which are upsampled to 512 feature channels, while doubling the spatial resolution, all the way back to 64 feature channels again. Each decoder have skip connection relations with its corresponding encoder block. This skip connection brings back the higher resolution spatial information that may be lost during encoding. The final output layer applies a 1×1 convolution to map the remaining 64 feature channels into the required number of target variables v .

2. GCN

The two Graph Convolutional models presented in this paper are a simpler GCN and a more complex Weighted Chebyshev Graph Convolutional Network, referred to as WeightedChebGCN in the code. Both models are implemented using the PyTorch Geometric packages `GCNconv` and `Chebconv`, respectively, each consisting of three convolutional layers. ReLU activation functions are applied for the first two layers, while a linear function is used as the final layer. The graph structure is constructed during a preprocessing step using PyTorch.

A key benefit of the graph representation is that nodes can communicate information to their nearest neighbors. In both models, the neighborhood is defined such that each node communicates with its 8 immediate neighbors: those immediate to the left and right, and those along the diagonals. In the GCN, all neighboring nodes contribute equally to the aggregation. In the WeightedChebGCN, a weighting function is applied to differentiate the contributions. The weight of immediate neighbors is set to 1, while the contribution from diagonal neighbors is scaled by their distance, computed by using a Gaussian Distance function. The Gaussian weight is calculated as a function of the Euclidean distance between neighboring nodes divided by a parameter σ .

In addition to the weighted neighborhood, the WeightedChebGCN is more complex in that the largest eigenvalue λ_{max} of the Graph Laplacian is estimated directly from the data using a PyTorch Geometric transformation function, to ensure consistent spectral scaling (PyTorch, 2026b). Several values of the K-localization parameter were also tested to determine how far information should propagate through the graph. The GCN does not include this feature and uses the simplified first-order approximation, equivalent to $K = 1$ in the Chebyshev approximation described in (Kipf and Welling, 2017).

During development, several variants of the Chebyshev model were tested. These include a deeper architecture matching the depth of a UNet, a non-weighted ChebGCN, and a periodically padded ChebGCN motivated by the assumption of a periodic turbulent system. None of these variants improved performance metrics and are therefore not included in the main report. However, they are considered highly relevant for further work in the context of more advanced datasets.

3. FNO

The FNO model used in this project follows the general architecture of the original FNO (Li *et al.*, 2021) architecture. It consists of a lifting operator P , four hidden layers, and a projection operator Q . In addition, the model includes three custom components: a local branch in the Fourier block, temporal positional encoding, and residual prediction.

After the temporal positional encoding, the input is passed through the lifting operator P implemented as a 1×1 convolutional layer.

The input has been lifted into a higher-dimensional hidden representation by the operator P . The feature channels are passed through a spectral convolution block, where they are transformed by a 2D FFT. The $K_{max} = K_x, K_y$ number of low-frequency modes are retained, while the remaining modes are truncated. In this implementation, both spatial dimensions are set to $K_x, K_y = 32, 32$. Secondly, the retained Fourier modes

are multiplied by the learned weight matrix R and are transformed back using an IFFT.

In parallel, the skip connection applies a pointwise convolution with a 1×1 kernel corresponding to the local linear transformation W and an additional customized local component that applies a 3×3 convolution with padding. The outputs from the spectral blocks, skip connections, and the local convolution component are summed together and passed through the non-linear activation function GELU.

These operations are repeated through the four FNO blocks. Lastly, the projection operator Q uses a 1×1 convolutional layer to map the hidden features back to its physical variables.

4. Simple moving average

The final model is a simple moving-average baseline (SMA), with the purpose of evaluating how well a simple non-learning method performs compared with complex and sophisticated deep learning models.

The baseline model uses the same temporal window structure as the other models. However, the next prediction is computed by taking the average of the four previous time steps. After each prediction, the input window is shifted forward by one step, the oldest frame is removed, and the previous prediction is appended as the most recent time step.

H. Training loop/Hyperparameter selection

Optimizer

Each of the network parameters was optimized using the stochastic optimization algorithm Adam, which employs an adaptive learning rate (Kingma and Ba, 2017). Adam was selected due to its popular use as an optimizer in previous studies focusing on physics-informed deep learning methods for weather and ocean forecasting (Furkan Tekin *et al.*, 2024; Galea and Ma, 2025; Manucharyan *et al.*, 2020; Wang *et al.*, 2020). All hyperparameters in Adam, except for the learning rate, were set to their default values

($\beta_1 = 0.9$, $\beta_2 = 0.999$), since these hyperparameters typically have limited impact on performance and computational resources were constrained (Kingma and Ba, 2017). The objective of the optimization process is to iteratively update all initialized weights and biases in the deep learning network to minimize the loss function. The loss functions used are described in Section II.H.

While the model architectures for the four models introduced in this report differ from each other, the training loops share a few common features, described in the following subsection.

Scheduled Sampling

Autoregressive models are prone to accumulating the error along the forecast horizon, due to inconsistencies between training and inference (Bengio *et al.*, 2015). A scheduled sampling technique is therefore applied to gradually transition the model from relying on supervised predictions to using its own outputs as input.

Several configurations have been tested to find the right trade-off. We use a scheduled sampling ratio $\epsilon \in (0, 1)$, where ϵ is reduced during training after a warm-up period with a given number of epochs (Bengio *et al.*, 2015). When $\epsilon = 1$, the model is trained exclusively on its ground truth, and as $\epsilon = 0$, the model is trained using its own inference results.

Learning Rate Scheduler

Similar to scheduled sampling, a learning rate scheduler is used to optimize the model training by dynamically adjusting the learning rate hyperparameter during training (PyTorch, 2026a). The scheduler helps the model converge faster and improves model stability (Li *et al.*, 2026). By controlling the step size used to minimize the loss during training, the model can take larger steps early in training for efficiency, while taking smaller steps later for more precise optimization.

Temporal Positional Encoding

A key challenge when working with time series data is ensuring that the model understands the temporal sequence of the input. To address this, a temporal positional encoding module is applied across all models before the convolutional layers. By adding a weighted bias to the input channels, the model is able to capture the temporal order of the data sequence.

Delta forecasting

Training ML models is computationally expensive due to the mathematical operations performed during optimization. One way to reduce this cost is delta forecasting ΔML (Grumet *et al.*, 2024). Rather than predicting absolute values, δML trains the model to predict differences between the current state and the next, and uses these changes for further predictions. This approximation simplifies the calculations and thereby reduces the associated computational cost (Grumet *et al.*, 2024).

Gradient Clipping

To avoid encounters with exploding gradients during backpropagation, we introduce gradient clipping to our models (PyTorch, 2026c). In this process, the gradients are rescaled if they reach the threshold value of 1 (Hjorth-Jensen, 2023).

Hyperparameters

The results presented in Section III are based on a set of hyperparameters that are kept constant across all four models. After testing various values and configurations to find the best trade-offs, the values listed in the table II were collectively agreed upon for the final training runs, to ensure consistent comparisons.

All models were trained using an early stopping mechanism, where training was terminated after a specified number of epochs, without performance improvements beyond a given threshold. This approach effectively reduces training time while also helping to prevent overfitting (Rasp and Lerch, 2018).

Hyperparameter	Value
Learning Rate (λ_{lr})	0.001
Learning Rate scheduler	5 epochs
Scheduled Sampling Ratio	0.6 (60%)
Scheduled Sampling warm up	5 epochs
Gradient penalty (λ_{grad})	0.075
Weighted penalty (λ_{σ})	0.075
Physics constraint (λ_{phys})	0.05
Gradient Clipping	1
Window Length	4
Forecast Length	4
Batch Size	32
Early Stopping Patience	10 epochs

TABLE II: Constant hyperparameters used across all results.

Loss functions

One of the main objectives of this project is to investigate how physics-informed loss functions can improve the ability of ML models to emulate physical behavior in turbulent systems. By focusing on five different loss functions, each tested across the four models presented in the report, we aim to identify which loss functions contribute to better forecasting performance and more accurate evaluation metrics. The motivation behind these loss functions is to construct a joint loss function that considers both vertical layers of the dataset, rather than using two unrelated losses that might fail to preserve important properties of the forecasting field (Hjorth-Jensen, 2023).

The first loss function is the MSE, which is used as a baseline and is given by Equation 29

$$MSE = \frac{1}{n} \sum (Y_i - \hat{Y}_i)^2 \quad (29)$$

Ideally, the value of the MSE should be zero, as the function measures the squared error between the target and predicted variables. Small values close to zero indicate that the predictions fit the ground truth well (Hjorth-Jensen, 2023).

To gradually increase the complexity of the loss functions, we included two modified versions of the original MSE function, which are a weighted MSE and a spatial gradient MSE. The weighted MSE is particularly relevant when working with this three-dimensional model, since the training set contains two vertical layers. Weighting is necessary to prevent one field from dominating the loss function, which may occur if one field has a substantially larger magnitude than the other (Hjorth-Jensen, 2023). To weight the function, we define two weights as $\alpha = \frac{1}{\sigma_1^2}$ and $\beta = \frac{1}{\sigma_2^2}$. The standard deviation of each layer acts as a normalization factor, allowing the two layers to be scaled comparably when applying the loss function.

Additionally, we introduced a spatial gradient MSE to reduce excessive smoothing in the forecast field (Hjorth-Jensen, 2023) as given in Equation 30. While retaining the original MSE term, an additional spatial gradient penalty term is included to preserve the field structure. Predictions that are close to the mean are penalized in order to maintain variability in the model output. The spatial gradient term includes both temporal and spatial gradients, as well as the vertical levels, v .

$$\mathcal{L} = \mathcal{L}_{data} + \lambda_{\nabla} \mathcal{L}_{\nabla} \quad (30)$$

Where $\mathcal{L}_{\nabla}$ is defined as:

$$\mathcal{L}_{\nabla} = \sum_{t,x,y,v} |\nabla \hat{h}_t^c(x,y) - \nabla h_t^c(x,y)| \quad (31)$$

In the quasi-geostrophic turbulent system represented in the training data, potential vorticity should be conserved (Srinivasan *et al.*, 2024). The model should therefore be capable of redistributing potential vorticity within the system without adding or removing vorticity, in accordance with conservation laws. To account for this, a soft physical constraint was introduced through the use of a physics-informed loss function ((Hjorth-Jensen, 2023) (Yan *et al.*, 2025)). The physically constrained loss function is formulated as in Equation 32.

$$\mathcal{L} = w_1 \mathcal{L}_1 + w_2 \mathcal{L}_2 + \lambda_{phys} \mathcal{L}_{phys} \quad (32)$$

Lastly, we tested a combined physics-based loss function, in which all previously introduced loss functions were included. The performance of the different loss functions is illustrated in Table III, while the results are discussed further in Section III.

III. RESULTS AND DISCUSSION

Interpreting results

Table III displays key metrics across all four models trained on five loss functions, compared to a moving average baseline. The metrics include per-step RMSE for each forecast step and an accumulated RMSE score, with the best score per model highlighted. The table is ordered by accumulated RMSE, with the moving average baseline at the top, followed by the four models ranked from best to worst.

FNO and UNet demonstrate strong predictive performance at the first forecast step $t + 1$, but their RMSE scores increase substantially from $t + 2$ to $t + 4$. Across all loss functions, accumulated RMSE scores remain similar, though weighted MSE performs best for FNO, while the combined physics-informed loss function yields the lowest accumulated RMSE for UNet.

Both FNO and UNet score approximately 0.2 higher than the graph convolutional models. However, their RMSE score decreases more gradually across each forecast step. Overall, both GCNs perform poorly throughout the forecast horizon, with the spatial gradient penalty MSE being the best-performing loss function for both.

From the table alone, FNO and UNet clearly outperform WeightedChebGCN and GCN, while all models outperform the baseline model. The following sections examine each model, using its best-performing loss function.

Figure 5a shows the RMSE for each channel and forecast step. The GCN achieves stable RMSE scores across all channels, and while FNO and UNet begin with lower RMSE, all models converge around forecast $t+3$. Because the GCN models capture less high-frequency structures, they remain numerically stable, but likely underfit the data relative to FNO and UNet.

This behavior is seen in Figure 2, which shows training and validation loss per epoch. Note that due to early stopping, the number of epochs varies across models. For FNO and UNet, there is a significant gap between validation and training loss, suggesting overfitting, with convergence occurring near epoch 6 for FNO and epoch 10 for UNet. In the early epochs, both losses decrease together, indicating an initially good fit. This also highlights the importance of early stopping to prevent further overfitting.

The GCNs show different trends. The gap between validation and training loss is considerably smaller, but both curves remain relatively flat after 10-15 epochs. The WeightedChebGCN shows validation loss improvement up to epoch 90, suggesting marginal improvements beyond the early stopping threshold. Despite the flat curve, the initial dip indicates an initially good fit. The following stagnation is likely due to underfitting, insufficient data complexity, or architectural limitations.

For the GCN, the training loss decreases immediately

TABLE III: Table containing training metrics. The best Accumulated RMSEs are highlighted in the table. All four models are tested against 5 chosen loss functions, and the following metrics are included: **RMSE** per timestep within a sample, **ARMSE**; Accumulated Root Mean Squared Error, **MAE**; Mean Absolute Error, and **MASE**: Mean Absolute Squared Error.

Model	Loss Function	RMSE $t+1$	RMSE $t+2$	RMSE $t+3$	RMSE $t+4$	ARMSE	MAE	MASE
Baseline								
	Moving Average	1.071	1.155	1.190	1.196	1.154	0.913	0.961
FNO								
	MSE (baseline)	0.634	0.910	0.984	0.999	0.894	0.690	0.726
	Weighted MSE	0.633	0.899	0.978	0.997	0.889	0.687	0.723
	Spatial Gradient	0.634	0.902	0.979	0.997	0.890	0.688	0.724
	Mean Constraint	0.639	0.917	0.993	1.004	0.900	0.695	0.731
	Combined Physics	0.630	0.901	0.981	0.998	0.890	0.687	0.723
UNet								
	MSE (baseline)	0.665	0.908	1.003	1.035	0.914	0.708	0.745
	Weighted MSE	0.646	0.897	1.000	1.035	0.907	0.701	0.737
	Spatial Gradient	0.641	0.898	0.999	1.032	0.906	0.699	0.735
	Mean Constraint	0.678	0.908	0.991	1.015	0.908	0.705	0.742
	Combined Physics	0.624	0.900	1.006	1.029	0.904	0.695	0.731
WeightedChebGCN								
	MSE (baseline)	0.869	0.971	0.989	0.994	0.961	0.758	0.729
	Weighted MSE	0.889	0.979	0.994	0.996	0.970	0.765	0.736
	Spatial Gradient	0.865	0.970	0.989	0.993	0.960	0.756	0.728
	Mean Constraint	0.866	0.971	0.989	0.993	0.961	0.757	0.728
	Combined Physics	0.872	0.973	0.989	0.994	0.963	0.759	0.731
GCN								
	MSE (baseline)	0.930	1.007	1.014	1.014	0.996	0.787	0.759
	Weighted MSE	0.950	1.012	1.018	1.017	1.004	0.793	0.764
	Spatial Gradient	0.929	1.005	1.014	1.014	0.995	0.786	0.757
	Mean Constraint	0.929	1.005	1.014	1.014	0.995	0.786	0.757
	Combined Physics	0.934	1.008	1.013	1.014	0.997	0.788	0.759

while the validation loss initially increases, which is a typical sign of overfitting. It is notable that this behavior appears early in the training. When interpreted together with the forecasts in Figure 6, it is clear that the GCN is underperforming. Visible artifacts appear already at $t+1$, and by $t+2$, spatial blurring dominates the forecast. The WeightedChebGCN produces forecasts resembling the forecasts of FNO and UNet, but still exhibits reduced variance throughout the forecast steps.

Spatial smoothing is evident across all four models, but progresses at different lead times. UNet best retains amplitude and variance in the forecast field, and at $t+4$ it still produces forecasts with defined high-amplitude regions. However, given its RMSE scores, the UNet appears to preserve spatial structure at the cost of accuracy, suggesting it may be reproducing plausible fields rather than emulating the dynamical state. Figure 5b quantifies this behavior through the amplitude collapse metric across the forecast horizon. The y-axis shows the ratio of the predicted standard deviation to the target standard deviation, where an ideal value of 1 indicates variance preservation. A threshold of 0.5 is used to remark a sig-

nificant collapse in the forecasts.

At $t+1$, both FNO and UNet remain above this threshold across all channels, with FNO reaching values above 0.8. Weighted ChebGCN and GCN fall below 0.5 at $t+1$ for ψ_{layer2} and partly q_{layer2} . FNO experiences a rapid decay across all channels, converging towards GCN values by $t+3$. UNet, by contrast, shows a gradual and consistent decline and becomes the best-performing model in terms of preserving variance from $t+2$ and onward.

The amplitude collapse is a concise metric for illustrating the dissipation of variance across the forecast horizon, which is what we see in Figure 6. Spatial smoothing is a well-known consequence of MSE-based loss functions (Yan et al., 2024) (Lam et al., 2023) (Nipen et al., 2025) (Lang et al., 2024)), thus we propose testing alternative loss functions that deviate from the spatial constraints induced by MSE, as discussed in Section III.

The results indicate that FNO and UNet are the two best-performing models, each with various strengths. FNO achieves lower RMSE scores, but produces lower variance in its forecast field and develops visible artifacts beyond $t+2$. UNet retains higher variance and exhibits

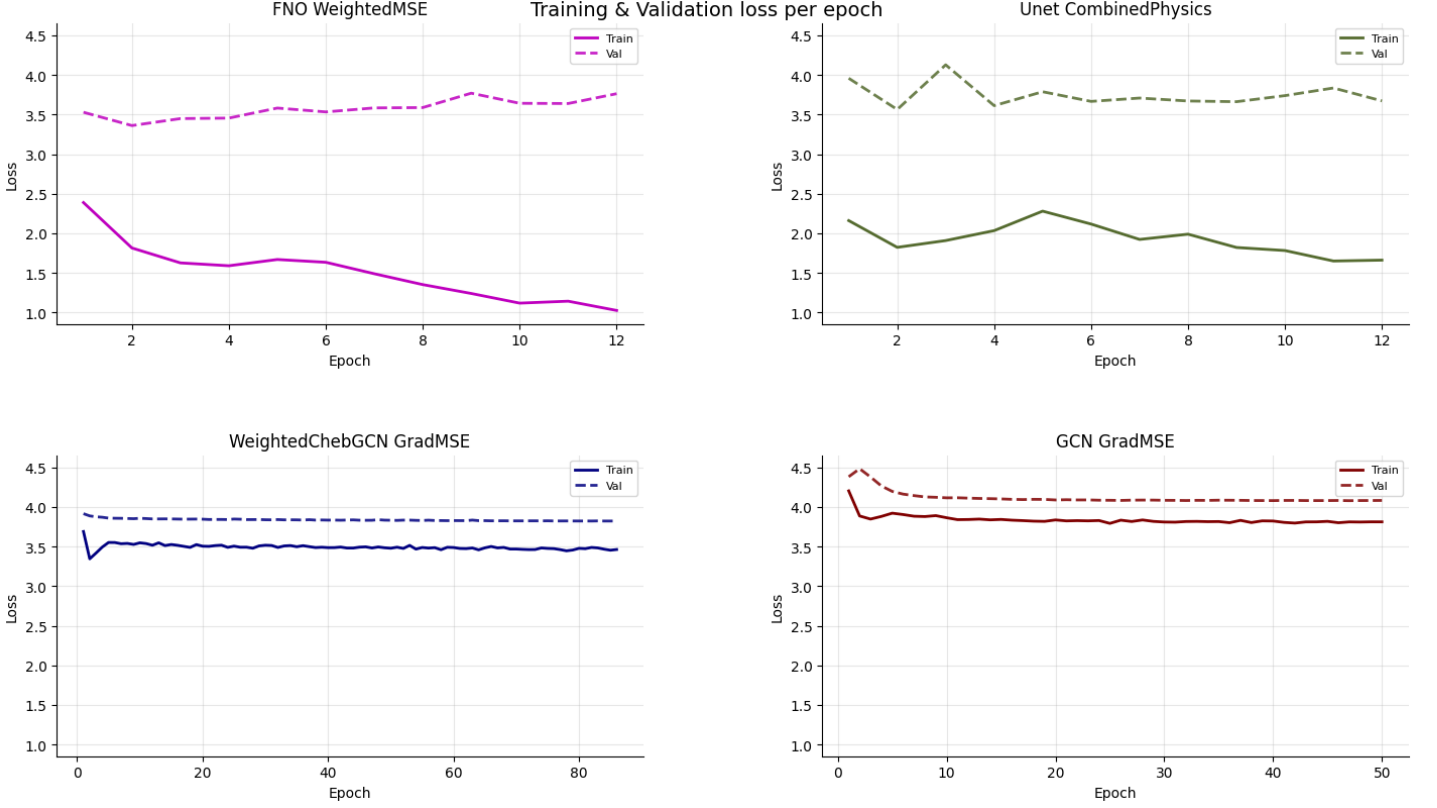


FIG. 2: Training and validation loss across all models. The x-axis ranges between 0 and 100 epochs, depending on when early stopping was activated during training.

less spatial smoothing, but its predicted values deviate more from the ground truth. The loss of variance in UNet is primarily near the edges, which may be a consequence of the combined physics-informed loss that penalizes the model near the boundary.

For the GCN and WeightedChebGCN, the results suggest architectural limitations. Two plausible explanations are: The 8-node neighborhood may be too restricted for information to propagate far enough to accurately capture the system’s dynamics, or the graph-based architecture may be unnecessarily complex relative for this regular grid dataset. While Graph Neural Networks and Graph Convolutional Networks have achieved competitive results in geophysical applications, it’s primarily in settings with irregular coastlines, bathymetry, and complex topography. Because the data here is defined on a regular grid, the architectural complexity of GCNs may not be ideal, and the models may consequently fail to emulate the approximated dynamics as expected.

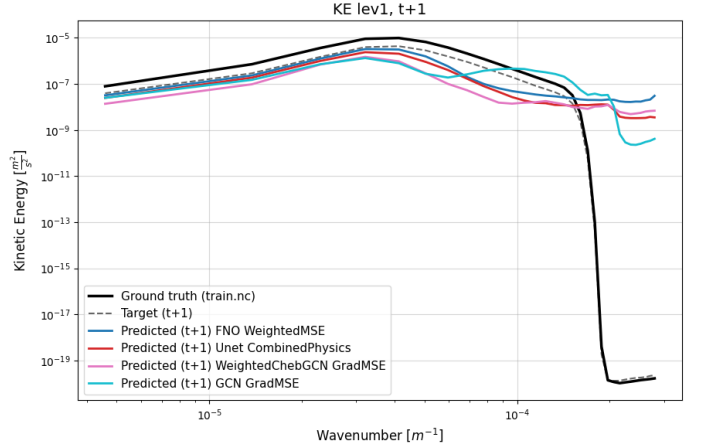


FIG. 3: The Kinetic Energy spectrum for the best four models. The black line is the Kinetic Energy directly calculated from the training dataset, and the grey dotted line is the ground truth calculated from the test data.

Figure 3 shows the kinetic energy distribution across wavenumbers K . According to Kolmogorov’s turbulence theory, the energy spectrum should follow a $k^{-\frac{5}{3}}$ decay in the inertial range, with a sharp drop towards higher wavenumbers in the dissipation range (Berge *et al.*,

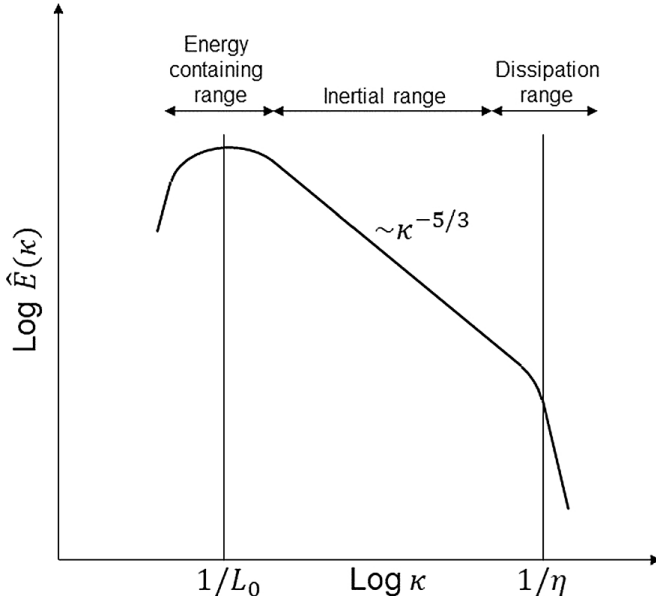


FIG. 4: Kolmogorov theory for the kinetic energy spectrum in Fourier space for non-linear Turbulent dynamics. Energy is ejected into the system at a small wavenumber, and is removed through dissipation towards larger wavenumbers (Berge *et al.*, 2026). Figure collected from the non-linear dynamics journal (Kalmár-Nagy and Bak, 2019).

2026), as illustrated in Figure 4. The models are therefore expected to fall between the ground truth and the idealized Kolmogorov curve.

All models follow the ground truth closely at low wavenumbers, but deviate towards the higher wavenumbers. The ground truth and target both have a small energy kickback near the high-wavenumber tail. This is an artifact discussed in (Yan *et al.*, 2025), and is likely due to excess energy accumulation in the offline model. None of the models reproduces the sharp dissipation drop-off expected from the Kolmogorov theory. Instead, all of the models exhibit excess energy accumulation towards the right tail. The GCN replicates the kickback artifact that’s seen in the ground truth and target, which is physically inconsistent with the turbulence theory (Yan *et al.*, 2025). The WeightedChebGCN and UNet show a slight kickback, though considerably less pronounced, making their energy spectra more physically plausible. The FNO shows no kickback at all and may have captured the energy spectrum in a manner that is more physically consistent than the target and ground truth. The WeightedChebGCN and GCN, however, deviate increasingly with the forecast length, underestimating the energy budget across both the internal and dissipation range. This is consistent with their tendency to smooth out high-frequency content.

Despite the issue of excess energy at high wavenum-

bers, indicating that all models struggle to accurately predict the dissipation range, the FNO produces the most physically reliable kinetic energy spectrum overall.

Observations in training and inference

The following observations were made during model exploration and tuning, with particular focus on scheduled sampling and delta forecasting.

One main challenge during model tuning was selecting appropriate values for scheduled sampling and its warm-up period. The purpose of a scheduled sampling is to reduce the discrepancy between training and inference by gradually exposing the model to its own predictions during training, thereby improving the model’s robustness and handling its own errors.

During the experimentation, a trade-off was found between forecast quality and amplitude preservation. Increasing the ratio for scheduled sampling improved both qualitative and quantitative results in terms of evaluation metrics. However, it resulted in a reduction in amplitude for later time steps as shown in Figure 5b. Higher scheduled sampling appeared to increase amplitude reduction, likely because forcing the model to use its own predictions inside the input window while training.

A possible explanation is that the insertion of predicted time steps into the next input window introduced an accumulated uncertainty from the turbulence dataset noise and the model’s own prediction errors. This may result in smoother predictions, due to an increasing error and biasing the model towards predicting the mean for each next time step, and eventually collapsing.

Despite these limitations, the model benefited from scheduled sampling by improving the prediction ability in an iterative free-run rollout. Without the exposure to its own predictions inside the training, the model was found to perform less robustly. Lower scheduled sampling also preserved amplitude but produced spatially less coherent and less accurate forecasts.

The final configuration used in this project favored a model with ability to forecast stronger short-horizon predictions with more visually coherent spatial structures, and an eventual reduction in amplitudes over later time steps, rather than preserving a stable amplitude across the full four time steps at the cost of poorer predictive accuracy.

A related, but distinct effect on predicting amplitudes concerns the use of the delta forecasting module, described in Section II.H. It not only appeared to provide a reduction in cost and simplification of calculations for both training and inference, but also to preserve the prediction amplitude closer to the target values.

Based on empirical experience, models without the delta forecasting module were required to reconstruct the entire grid from scratch, including both amplitudes and

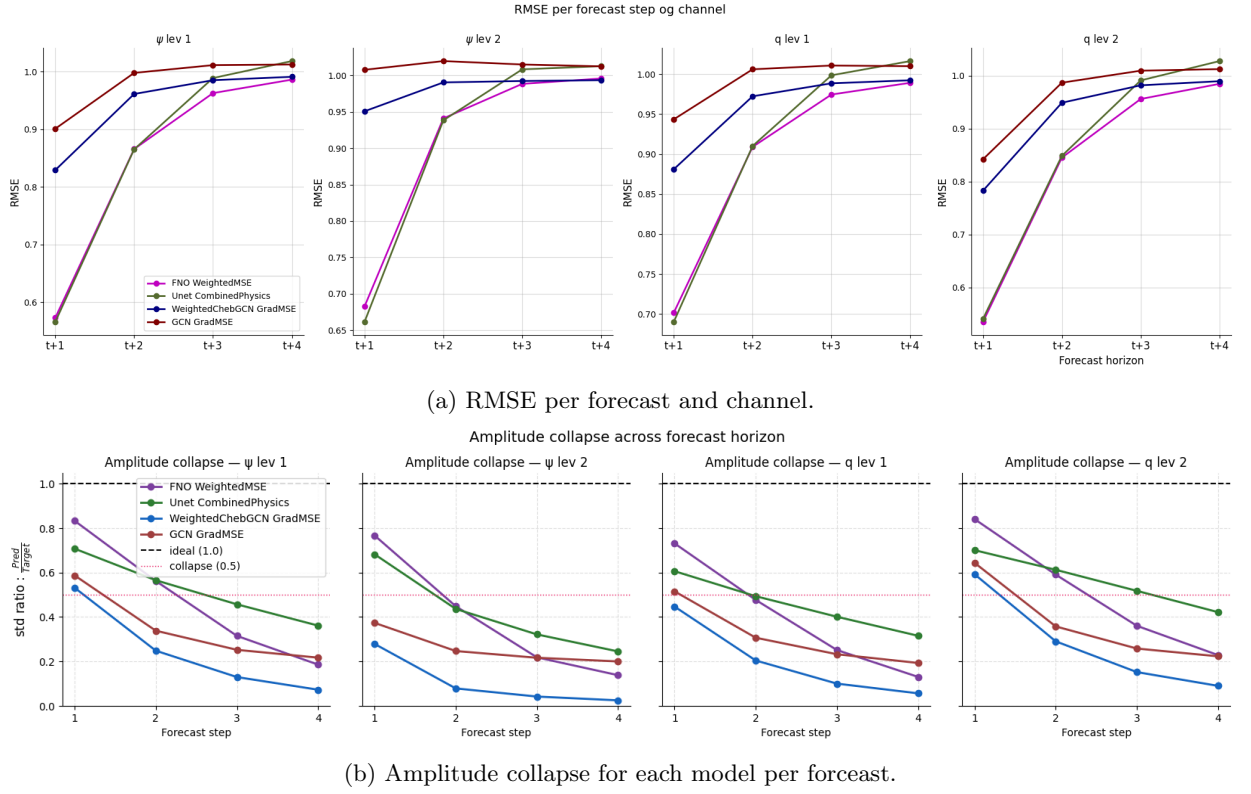


FIG. 5: Evaluation results for the best four models: (a) amplitude collapse and (b) RMSE per forecast and channel.

spatial structures. This appeared to place a heavier burden on the network and resulted in weaker amplitude predictions, even for the first predicted time step $t + 1$.

After adding the delta forecast module, the model only needed to learn the differences between consecutive states, rather than reconstructing the entire field. As a result, the predictions retained stronger amplitudes, notably just after the first timestep, but also across longer horizons. However, the reduction of amplitude for later time steps still remained a persistent challenge for the model.

The primary structural downside of delta forecasting is an increased reliance on persistence. By anchoring each prediction to the previous timestep, the model is implicitly biased towards producing outputs closer to the prior state. This disadvantage might be most critical for real-world scenarios, where the monitored field amplitudes can change abruptly, making it a meaningful limitation. However, for the closed quasi-geostrophic turbulence simulation system, this persistence bias appeared to be beneficial, as physically related states evolved gradually and large abrupt changes were uncommon.

Compare to prior research

For an external comparison, the results obtained in the present project can be related to prior studies on autoregressive deep learning models predicting future states of dynamical physical systems.

The closest reference is the study by Chattopadhyay et al. (Chattopadhyay *et al.*, 2020) who used a spatial transformer-based convolutional encoder-decoder model. The study predicted the upper-layer streamfunction ψ in a two-layer quasi-geostrophic turbulence system without having access to its lower layer. Their dataset had a spatial resolution of 88×128 grid points and a temporal resolution of one day. The qualitative results are shown in Figure 12.

A key result from the study by Chattopadhyay et al. is that the use of a spatial transformer module and custom loss functions significantly improved the autoregressive rollout performance of their deep learning model. The proposed method reduced the degradation of model predictions with an increasing forecast horizon, compared to the baseline model. However, the study does not explicitly quantify the amplitude collapse as in the present study. Visually, the model predictions stay consistent for the first five shown time steps in Figure 12.

Although their qualitative predictions are promising, they only cover a short prediction horizon of 5 days. Their model appears to outperform its baseline models by

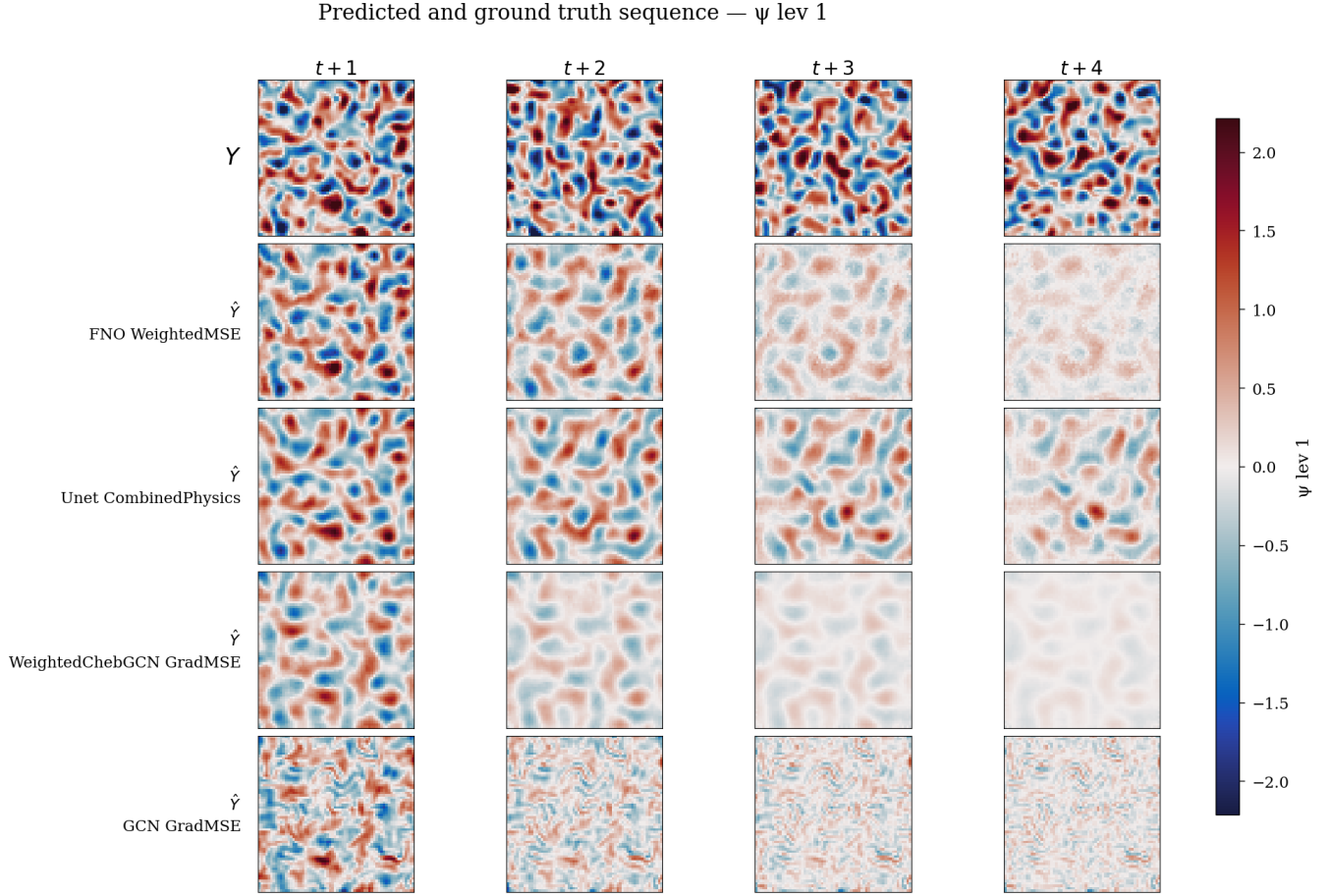


FIG. 6: Predicted & ground truth sequence for ψ lev 1 of the dataset. Sample 100 with the four best-performing loss functions of each ML model. The top sequence is the ground truth test set (rewrite later).

10% for the first 9 forecast days, as shown in Figure 13. For longer forecast horizons, the model seems to enter a state of more variable predictions by monitoring their evaluation metric: the point-wise correlation coefficient between two time steps, denoted as R . The R decreases from approximately 0.98 at prediction $t + 1$ down to approximately $R \approx 0.45$ after a forecast horizon of only 15 days. After this decrease, R fluctuates between 0.45 and 0.9 for the remaining 75 days. This shows that autoregressive rollout models with an additional spatial transformer, to an extent, contribute to learning the physics, but still struggle to preserve and predict detailed long horizons of chaotic flow.

This behavior is consistent with the results observed in the present project. Showing a fast increase in error with rollout steps and an eventual big decrease in prediction capability for long-term predictions. This might be a possibility due to cumulative errors that are fed back to the model as input for the prediction of the next time step.

The comparison with Chattopadhyay et al. (Chattopadhyay et al., 2020) suggests that an additional spa-

tial transformation module could potentially mitigate the fast amplitude collapses observed in the present project. In the present project, such a mechanism could help the models learn the movement and deformation behaviors of vortices more effectively, through the two-dimensional affine transformation that includes: translation in the x - y plane, rotation, scaling, stretching, and shearing. In contrast, the FNO represents spatial evolution through spectral mode interaction and removal of several high-frequency Fourier modes, while the U-net tends to aggregate spatial dimensions by max pooling, as in the U-Net, possibly also potentially removing smaller-scale eddies. This might contribute to learning smoother predictions and the reduction of sharp, high-amplitude small-scale predictions for later rollout steps.

Another relevant study, that might be less directly comparable, but still relevant to geophysics forecasting, is the work by Tekin et al. (Furkan Tekin et al., 2024). The study investigates numerical weather prediction methods using several deep learning models. Their proposed model uses a stacked ConvLSTM unit together with customized extensions such as a context-matcher and an at-

tention mechanism. The method of predictions is similar and uses an iterative rollout for a three-hour temporal resolution of a 30 km spatial extent divided into 61×121 grid points. The given results are in Appendix Figure 14 (Furkan Tekin *et al.*, 2024). The data used in the study appear to consist of smoother dynamics and a stronger short-term persistence between consecutive time steps, compared to the quasi-geostrophic turbulence datasets used in the present project. The study compares their proposed model with recurrent neural networks such as LSTM, GRU, but also U-Net as baselines to capture spatial-temporal correlations. As discussed by Tekin *et al.* (Furkan Tekin *et al.*, 2024), their investigated models tend to capture successfully early fluctuations, but later the models focus on stationary regions (Furkan Tekin *et al.*, 2024).

Further, with an increasing forecast horizon, their predictions also become smoother, and the movement of large-scale structures no longer corresponds with the ground truth. The model, however, does not collapse as much as it can be seen in the present study, Figure 6.

The two-layered quasi-geostrophic baroclinic turbulence data used in the present study are very chaotic, with time steps of approximately 42 days apart. This large temporal sampling makes consecutive time steps highly decorrelated. In addition, the data grid has a very coarse spatial resolution for its big physical extent.

These differences make it difficult to be directly comparable with previous Chattopadhyay *et al.* (Chattopadhyay *et al.*, 2020) that have greater spatial and temporal resolutions and allow the models to learn strong correlations between consecutive time steps as they often are more similar. Similarly, the weather forecasting study of Tekin *et al.* (Furkan Tekin *et al.*, 2024), their weather forecasting model consists of smoother temporal evolution and a higher degree of persistence between the consecutive time steps for variables such as temperature forecasting. In contrast to the present project, which uses data with lower spatial and temporal resolution and high variability and low predictability due to temporal decorrelation in a turbulent ocean-like system.

Error analysis

The FNO model with a weighted MSE loss function has achieved the lowest accumulated RMSE, making it interesting for further error analysis. Two time steps have been chosen for comparison: the first prediction $t+1$ and the last $t+4$.

By comparing model predictions against their targets in a scatter plot, we can quantify and distinguish the model’s accuracy from its systematic errors or bias and the model’s precision from the variance.

First, focusing on the four upper plots in Figure 7, where the black diagonal lines indicate a theoretical per-

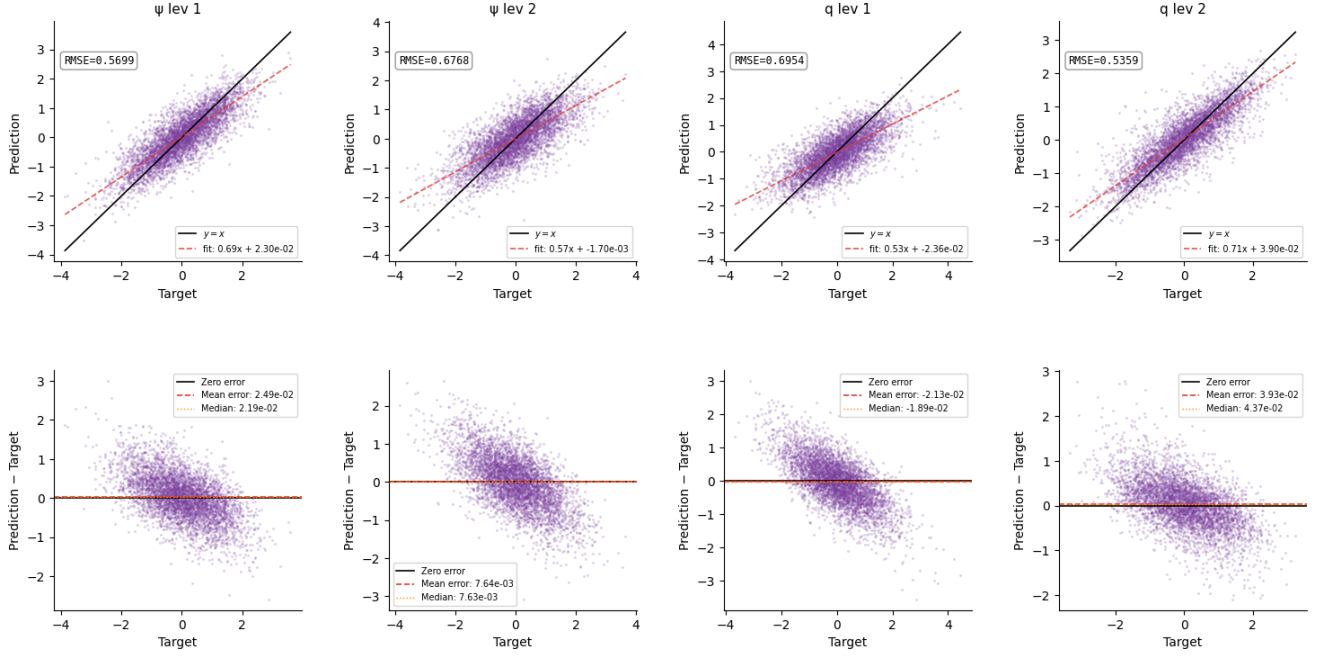
fect match between predictions and targets, and the red line is a linear fit of the model predictions against target values. The intercept term of the fitted red lines explains the model’s global bias, given in z-score normalized units, which range between -0.0017 and 0.039 for the different variables. Here, 0 indicates a theoretically unbiased model. In this case, with an average global bias of 0.00918, indicating that it is a relatively small error compared to the variance, were the point clusters are around the diagonal line, but remain greatly sparse. However the model seems to capture the general relationships for the first forecast step, especially well for the bottom-layer vorticity variable q with an RMSE value of 0.5359 as shown in Figure 7, while the upper-layer vorticity is the most difficult which is meaningful, as upper layer tend to have a higher degree of turbulence due to its higher speed, compared with the slower, deeper regions.

When comparing the first predicted step, $t+1$, with the two variables shown in the upper part of the plot for the last predicted time step $t+4$ in Figure 8. There is a common trend that appears across the variables. The model predicts close to zero, in the case of z-score normalization, zero is also the mean. This indicates the model capture little of the target’s explanatory variance. The predictions are of low variance and show weak dependence on the target values compared to the first predicted time step $t+1$. This type of behavior is comparable to a model predicting a Gaussian-like distribution centered around zero on the prediction axis based on the visual aspect in Figure 8 and the reported RMSE values close to one, approximately indicating a standard deviation of one for the z-score normalization.

When comparing the residual patterns from the four bottom scatter plots for residual/error against target value in Figure 7. Here, the black line indicates perfect predictions, the red fitted line in the bottom four plots indicates the mean trend of the prediction residuals, confirming the previously stated small global prediction bias for each variable. However, when interpreting this trend, there seems to be a conditional bias, rather than only a global bias. This is suspected due to the model’s scatter plots showing a main trend downward across the whole target axis. The model consistently predicts lower amplitude than the targets. This means that the model tends to overpredict negative targets while underpredicting positive targets, systematically dragging the amplitudes towards the mean.

This trend is even more emphasized by the later time steps of $t+4$ as shown in the four bottom plots in Figures 8. Here, the variance due to random error appears minimal due to the dense, narrow cluster that extends diagonally across the full range of possible target values across all variables. However, the bottom-layer of vorticity q seems again to be the most predictable for both time steps $t+1$ and $t+4$.

FNO WeightedMSE · t+1

FIG. 7: Scatter plot for FNO with weighted MSE loss function for $t + 1$.

Data

All the results presented in this study are run without an initial spin-up period, focusing instead on the inertial and dissipation ranges of the dynamics, as illustrated in Figure ???. The spin-up period, which lasts approximately 30 time steps in the dataset, is excluded to prevent the models from learning the initial behavior that doesn't generalize to the dynamics in the later ranges.

Each time step in the dataset corresponds to a period of 42 days, meaning each inference sample has a forecast length of 42 days. This is an unusually long time horizon for machine learning emulation of dynamical systems, and may partly explain why the models struggle to reproduce the underlying dynamics accurately. As previously mentioned in the introduction, the offline model is run at a low resolution due to time and computational constraints, which further limits the quality of the training data and, consequently, the results.

Turbulence modeling is nevertheless relevant across a range of use cases. Operational ocean and weather forecasting prioritizes transient features, which are changes in the atmosphere-ocean dynamics on shorter time-scales. Meanwhile, climate research tends to focus on long-term trends and day-to-day variability (Berge *et al.*, 2026). In this context, testing shorter time steps would be of interest, particularly to examine whether reduced time steps improve performance at longer lead times. That said, a 42-time step is not without interest, and is well-suited

for capturing larger-scale dynamics operating on longer time-scales.

Turbulence theory remains one of the fundamental challenges in geodynamics. The non-linear dynamics and their wide range of temporal and spatial scales make it difficult to accurately represent in forecasting systems. The Kinetic Energy spectrum, as discussed in the results, illustrates this challenge well 3. Correctly capturing energy distributions across all scales remains a challenge.

Limitations

While ML models offer great benefits, they also come with certain limitations. Training is computationally expensive, whereas inference is generally more efficient and associated with lower costs. One advantage of the quasi-geostrophic dataset used in this project is its relatively small size: a 64×64 grid with two vertical layers results in a dataset of 1.6 Gigabytes. Consequently, training costs have remained relatively low, although training times vary considerably between the models, as shown in 9. The graph-structured models require more memory and longer training times than the CNN and FNO models, which is expected given the increased complexity of their architectures. Inference, however, is significantly less computationally demanding and can be performed more efficiently across all four models.

Trade-offs are a central consideration in machine learn-

FNO WeightedMSE · t+4

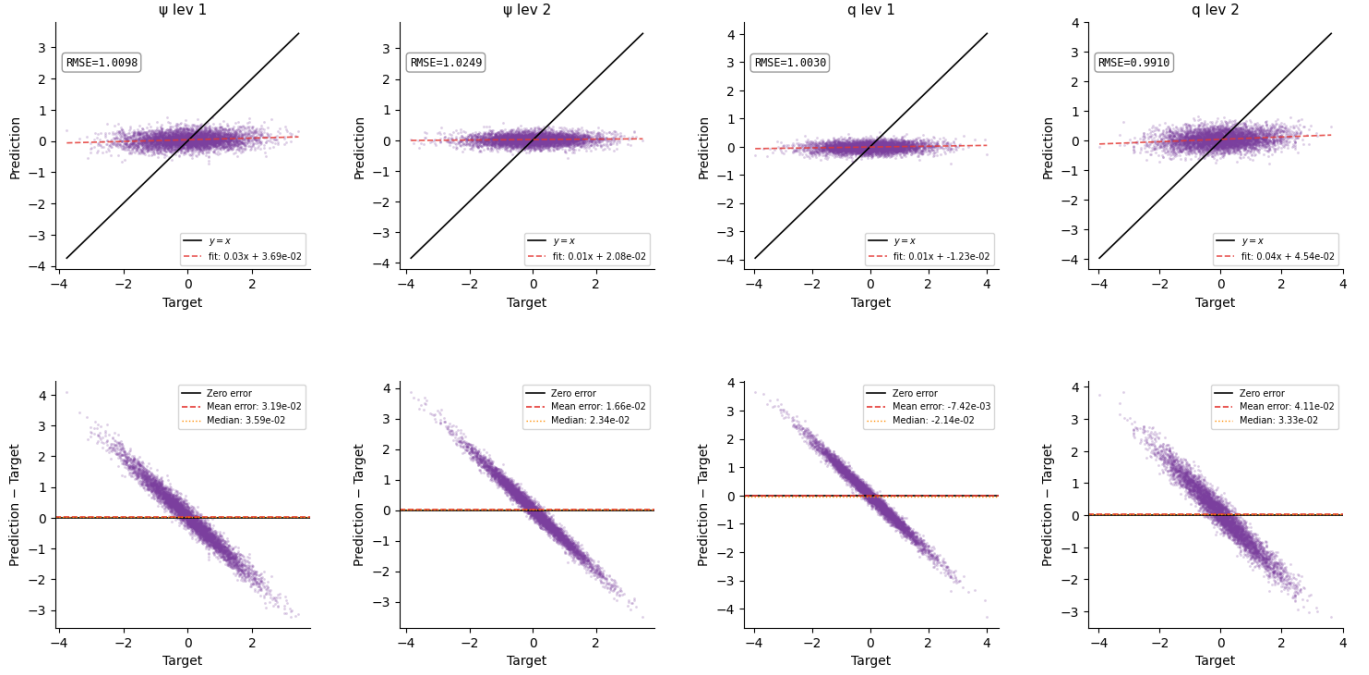
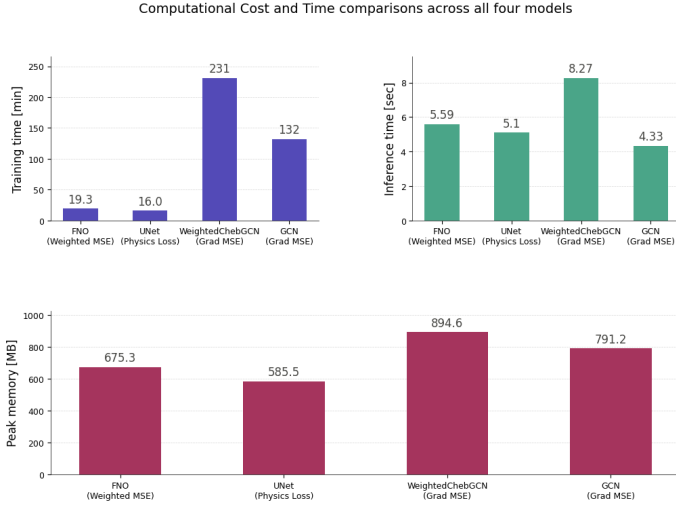
FIG. 8: Scatter plot for FNO with weighted MSE loss function for $t+4$.

FIG. 9: Time and Computational constraints across all four models.

ing. Although a smaller dataset reduces computational cost, it may also limit the information available for the model to learn the underlying physical patterns accurately. This is evident in the present work, where the limited dataset size may restrict the model's ability to capture the full dynamics of quasi-geostrophic turbulence.

It is worth noting that incorporating an online fluid

dynamical model into the training process could provide several advantages (Yan *et al.*, 2025). However, this report focuses on the offline setting, as integrating the dynamical model is beyond the scope of this work due to computational costs and time constraints.

An important question is therefore whether the models must explicitly learn the physical properties encoded through the online model, or whether approximating these properties through physics-informed loss functions is sufficient for turbulent forecasting. As noted in (Yan *et al.*, 2025), the offline approach treats forecasting as a regression task between inputs and outputs without modeling temporal correlations. Potential drawbacks of this approach include a lack of temporal awareness and uncertainty in whether soft physical constraints applied in physics-informed loss functions can provide the same stabilizing effect as the online dynamical model. The addition of temporal positional encoding is a further attempt to encourage the models to capture the temporal structure of the dataset.

Future work / Perspectives

Based on the findings of this study, several questions remain unanswered. We will therefore propose directions for future work that were outside of the scope of this report.

Testing alternative neural network architectures for handling temporal dependencies in the data would be of interest. Recurrent Neural Networks (RNNs) are a class of neural networks that incorporate backwards connections through a hidden state, allowing information to persist across time steps (Hjorth-Jensen, 2023). RNNs are commonly used for time-dependent tasks, and specific architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) are known for capturing long-range temporal dependencies. This would be particularly relevant to test on the quasi-geostrophic data.

Additionally, the predictions in this study correspond to a forecast horizon of 42 days. Reducing the time step could allow the models to capture finer dynamical changes in the system.

As discussed in the results section, all models produce spatially blurred predictions, a known consequence of the spatial constraint implicit in MSE—based loss functions. Testing loss functions that deviate from MSE and MAE formulations would be of particular interest. Given that three of the models represented in this report operate in the Fourier domain, a Fourier Amplitude loss function, where the loss is computed on the amplitude of each frequency band, could be a promising approach for recovering high-frequency structures in the forecast field.

IV. CONCLUSIONS

To explore data-driven long-term predictions of baroclinic turbulence in a quasi-geostrophic three-dimensional dataset, this report has examined whether deep learning models are able to emulate the associated dynamical behavior of turbulence. The results reveal certain strengths across the tested models. The FNO is able to emulate the physical approximation of the Kolmogorov energy spectrum in the inertial and dissipation range, while still accumulating errors across forecast steps and lacking high-frequency spatial structure in the predictions.

The loss of high-frequency variation is a common finding across all four models, where amplitude collapse occurs within forecast steps $t + 2$ and $t + 3$ for all architectures. This is likely due to a spatial smoothing induced by MSE-based loss functions. UNet retains the highest variance across all four forecast steps, though its RMSE scores remain relatively high, indicating that the predictions still deviate considerably from the ground truth, despite preserving spatial structure.

The unusually long time step of approximately 42 days per forecast sample may impact the results in several ways. While long-term dynamics are highly relevant for research purposes in the ocean-atmosphere dynamics, it could be beneficial to train on shorter time steps and evaluate performance at longer lead times.

The results from testing five different loss functions indicate that the loss function choice is not the primary driver of the performance, as the evaluation metrics remain nearly alike across all five functions. The autoregressive instability of the forecasts dominates the field, suggesting that the main limitations lie in architectural choices or model choices relative to the complexity of the data, rather than the loss function itself.

It is therefore of great interest to study alternative architectures in future work, such as recurrent neural networks designed for temporally dependent data, including LSTM and GRU variants. Should architectural improvements yield stronger results, this would be a natural first step before revisiting the question of whether physics-informed loss functions improve the emulation of baroclinic quasi-geostrophic turbulence. With an improved foundation, the introduction of non-MSE loss functions, such as the Fourier amplitude loss function, would be a motivating next step.

V. APPENDIX: USE OF AI/LLM TOOLS

For this project, Artificial Intelligence (AI) has been used for the following purposes:

- Help with code structure, debugging, and improving the efficiency of the code.
- Help to brainstorm ideas for model architectures and to summarize relevant research articles.
- AI has been used to help efficiently create validation plots.

TABLE IV: Overview of LLM contribution in report

Section	LLM contribution	Notes
Abstract	1	Grammar check only
Introduction	1	Grammar check only
Theory / Methods	1	Grammar check only
Results/ Discussion	1	Grammar check only
Conclusion	1	Grammar check only

For information about LLM use in the code, please refer to the GitHub page: https://github.com/Malenefk/Advanced_MachineLearning_Project/blob/main/LLM/LLM_use.txt

REFERENCES

- S. Adamov, J. Oskarsson, L. Denby, T. Landelius, K. Hintz, S. Christiansen, I. Schicker, C. Osuna, F. Lindsten, O. Fuhrer, and S. Schemm (2025), eng“Building machine learning limited area models: Kilometer-scale weather forecasting in realistic settings,” Swedish Publication Index.
- K. Azizzadenesheli, N. Kovachki, Z. Li, *et al.* (2024), “Neural operators for accelerating scientific simulations and design,” *Nature Reviews Physics* **6**, 320–328.
- S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer (2015), eng“Scheduled sampling for sequence prediction with recurrent neural networks,” Arxiv.
- E. Berge, Breivik, K. Christensen, J. Lacasce, and G. Sutherland (2026), *Turbulent boundary layers in the atmosphere and the ocean* (University of Oslo).
- A. Chattopadhyay, M. Mustafa, P. Hassanzadeh, and K. Kashinath (2020), “Deep spatial transformers for autoregressive data-driven forecasting of geophysical turbulence,” pp. 106–112.
- T. Chen, and H. Chen (1993), “Approximation capability to functions of several variables, nonlinear functionals and operators by radial basis function neural networks,” in *Proceedings of 1993 International Joint Conference on Neural Networks*, Vol. 2 (IEEE) pp. 1439–1442.
- T. Chen, and H. Chen (1995), “Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems,” *IEEE Transactions on Neural Networks* **6** (4), 911–917.
- B. X. Defferrard, Michael, and P. Vandergheynst (2016), eng“Convolutional neural networks on graphs with fast localized spectral filtering,” <https://arxiv.org>.
- S. Furkan Tekin, A. Fazla, and S. Serdar Kozat (2024), “Numerical weather forecasting using convolutional-lstm with attention and context matcher mechanisms,” *IEEE Transactions on Geoscience and Remote Sensing* **62**, 1–13.
- D. Galea, and H.-Y. Ma (2025), “Intercomparison of deep learning model architectures for atmospheric river prediction,” *Artificial Intelligence for the Earth Systems* **10.1175/AIES-D-24-0057.1**, published online 27 June 2025.
- I. Goodfellow, Y. Bengio, and A. Courville (2016), *Deep Learning* (MIT Press) <http://www.deeplearningbook.org>.
- S. Goswami, A. Bora, Y. Yu, and G. E. Karniadakis (2022), “Physics-informed deep neural operator networks,” *arXiv:2207.05748 [cs.LG]*.
- M. Grumet, C. von Scarpatetti, T. Bučko, and D. A. Egger (2024), “Delta Machine Learning for Predicting Dielectric Properties and Raman Spectra,” *The Journal of Physical Chemistry C* **128** (15), 6464–6470.
- D. K. Hammond, P. Vandergheynst, and R. Gribonval (2009), eng“Wavelets on graphs via spectral graph theory,” *arXiv.org*.
- Havforskningsinstituttet (2026), no“Kyst og fjord,” .
- K. He, G. Gkioxari, P. Dollár, and R. Girshick (2018), “Mask r-cnn,” *arXiv:1703.06870 [cs.CV]*.
- M. Hjorth-Jensen (2023), *Computational Physics Lecture Notes* (Department of Physics, University of Oslo, Norway).
- K. Hornik, M. Stinchcombe, and H. White (1989), “Multilayer feedforward networks are universal approximators,” *Neural Networks* **2** (5), 359–366.
- T. Kalmár-Nagy, and B. D. Bak (2019), eng“An intriguing analogy of kolmogorov’s scaling law in a hierarchical mass–spring–damper model,” *Nonlinear dynamics* **95** (4), 3193–3203.
- R. Keisler (2022), eng“Forecasting global weather with graph neural networks,” Arxiv.
- D. P. Kingma, and J. Ba (2017), “Adam: A method for stochastic optimization,” *arXiv:1412.6980 [cs.LG]*.
- T. N. Kipf, and M. Welling (2017), eng“Semi-supervised classification with graph convolutional networks,” *arXiv.org*.
- N. Kovachki, S. Lanthaler, and S. Mishra (2021), “On universal approximation and error bounds for fourier neural operators,” *arXiv:2107.07562 [math.NA]*.
- N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar (2023), “Neural operator: learning maps between function spaces with applications to pdes,” **24** (1).
- R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu, A. Merose, S. Hoyer, G. Holland, O. Vinyals, J. Stott, A. Pritzel, S. Mohamed, and P. Battaglia (2023), eng“Learning skillful medium-range global weather forecasting,” *Science (American Association for the Advancement of Science)* **382** (6677), 1416–1421.
- S. Lang, M. Alexe, M. Chantry, J. Dramsch, F. Pinault, B. Raoult, M. C. A. Clare, C. Lessig, M. Maier-Gerber, L. Magnusson, Z. B. Bouallègue, A. P. Nemesio, P. D. Dueben, A. Brown, F. Pappenberger, and F. Rabier (2024), eng“Aifs – ecmwf’s data-driven forecasting system,” Preprint Arxiv.
- Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel (1989), “Backpropagation applied to handwritten zip code recognition,” *Neural Computation* **1** (4), 541–551, <https://direct.mit.edu/neco/article-pdf/1/4/541/811941/neco.1989.1.4.541.pdf>.
- B. Li, F. Chen, Z. Huang, L. Wang, and L. Wu (2026), “Functional scaling laws in kernel regression: Loss dynamics and learning rate schedules,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar (2020), “Neural operator: Graph kernel network for partial differential equations,” *arXiv:2003.03485 [cs.LG]*.
- Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar (2021), “Fourier neural operator for parametric partial differential equations,” *arXiv:2010.08895 [cs.LG]*.
- L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis (2021), “Learning nonlinear operators via deeponet based on the universal approximation theorem of operators,” *Nature Machine Intelligence* **3** (3), 218–229.
- G. E. Manucharyan, L. Siegelman, and P. Klein (2020), “A deep learning approach to spatiotemporal sea surface height interpolation and estimation of deep currents in geostrophic ocean turbulence,” *Journal of Advances in Modeling Earth Systems* **10.1029/2019MS001965**.
- T. N. Nipen, H. H. Haugen, M. S. Ingstad, E. M. Nordhagen, A. F. S. Salihi, P. Tedesco, I. A. Seierstad, J. Kristiansen, S. Lang, M. Alexe, J. Dramsch, B. Raoult, G. Mertes, and M. Chantry (2025), eng“Regional data-driven weather modeling with a global stretched-grid,” *Artificial intelligence for the earth systems*.
- J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Aziz-

- zadenesheli, P. Hassanzadeh, K. Kashinath, and A. Anandkumar (2022), “Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators,” [arXiv:2202.11214 \[physics.ao-ph\]](#).
- PyTorch (2026a), en“[LRScheduler — PyTorch 2.12 documentation](#),” .
- PyTorch (2026b), “[torch_geometric.transforms.laplacian_lambda_r — pytorch_geometric documentation](#),” .
- PyTorch (2026c), en“[torch.nn.utils.clip_grad_norm_ — PyTorch 2.12 documentation](#),” .
- S. Raschka, Y. H. Liu, and V. Mirjalili (2022), *Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python* (Packt Publishing Ltd).
- S. Rasp, and S. Lerch (2018), eng“Neural networks for post-processing ensemble weather forecasts,” *Monthly weather review* **146** (11), 3885–3900.
- O. Ronneberger, P. Fischer, and T. Brox (2015), “U-net: Convolutional networks for biomedical image segmentation,” [arXiv:1505.04597 \[cs.CV\]](#).
- K. Sofeikov (2025), en“[What is the physical meaning of graph Laplacian?](#)” .
- K. Srinivasan, M. D. Chekroun, and J. C. McWilliams (2024), en“Turbulence Closure With Small, Local Neural Networks: Forced Two-Dimensional and -Plane Flows,” *Journal of Advances in Modeling Earth Systems* **16** (4), e2023MS003795.
- E. Tziperman (1992), “Methods of testing parameterizations: Vertical ocean mixing,” in *Modeling the Earth System, Volume 3*, nTRS Author Affiliations: Weizmann Inst. of Science NTRS Document ID: 19940026128 NTRS Research Center: Legacy CDMS (CDMS).
- R. Wang, K. Kashinath, M. Mustafa, A. Albert, and R. Yu (2020), “Towards physics-informed deep learning for turbulent flow prediction,” [arXiv:1911.08655 \[physics.comp-ph\]](#).
- G. Wen, Z. Li, Q. Long, K. Azizzadenesheli, A. Anandkumar, and S. M. Benson (2023), “Real-time high-resolution CO2 geological storage prediction using nested fourier neural operators,” *Energy & Environmental Science* **16** (4), 1732–1741.
- C.-W. Yan, S. Q. Foo, V. H. Trinh, D.-Y. Yeung, K.-H. Wong, and W.-K. Wong (2024), eng“Fourier amplitude and correlation loss: Beyond using l2 loss for skillful precipitation nowcasting,” [Arxiv](#).
- F. Yan (2025), “Data collection for online learning in two-layer quasi-geostrophic model,” [10.5281/zenodo.15244460](#).
- F. E. Yan, H. Frezat, J. Le Sommer, J. Mak, and K. Otneess (2025), “Adjoint-based online learning of two-layer quasi-geostrophic baroclinic turbulence,” *Journal of Advances in Modeling Earth Systems* **17** (7), e2024MS004857, e2024MS004857 2024MS004857.
- S. Zhang, H. Tong, J. Xu, and R. Maciejewski (2019), eng“Graph convolutional networks: a comprehensive review,” *Computational social networks* **6**, 1–23.

Appendix A: Appendix

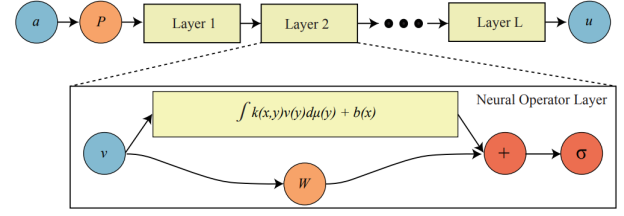


FIG. 10: Original architecture for a Fourier neural operator. Here, a denotes the input function, P denotes the lifting operator, Layer 1, Layer 2, and Layer L denote the given layers in the Fourier neural operator. u denotes the output function, obtained after a final projection back to the original space. In each Fourier neural operator layer, there is a v denoting the current latent representation, with parallel operation: integral kernel operator with a bias denoted by $b(x)$ and a point-wise linear transformation W , acting as a skip connection. The red $+$ denotes summing, before passing into a non-linear activation function denoted as the red sigma. The Figure is adapted and modified from Li et al. (Kovachki *et al.*, 2023).

Par.	Value	Unit	Description
nx	64	–	Grid points in x
ny	64	–	Grid points in y
nz	2	–	Vertical layers
dt	3600	s	Time step (10 model years)
$tmax$	311040000	s	Total duration
$taveint$	86400	s	Averaging interval
$twrite$	1000	s	Output interval
L	1000000	m	Domain length
W	1000000	m	Domain width
rd	15000	m	Deformation radius
$delta$	0.25	–	Layer thickness ratio
$del2$	0.80	–	Dissipation parameter
rek	$5.787e-7$	–	Bottom drag
$beta$	$1.500e-11$	–	Vorticity gradient

TABLE V: Main simulation parameters retrieved from the two-layer quasi-geostrophic baroclinic turbulence system metadata. Original dataset had 256×256 .

Comperison of data before and after spin-up

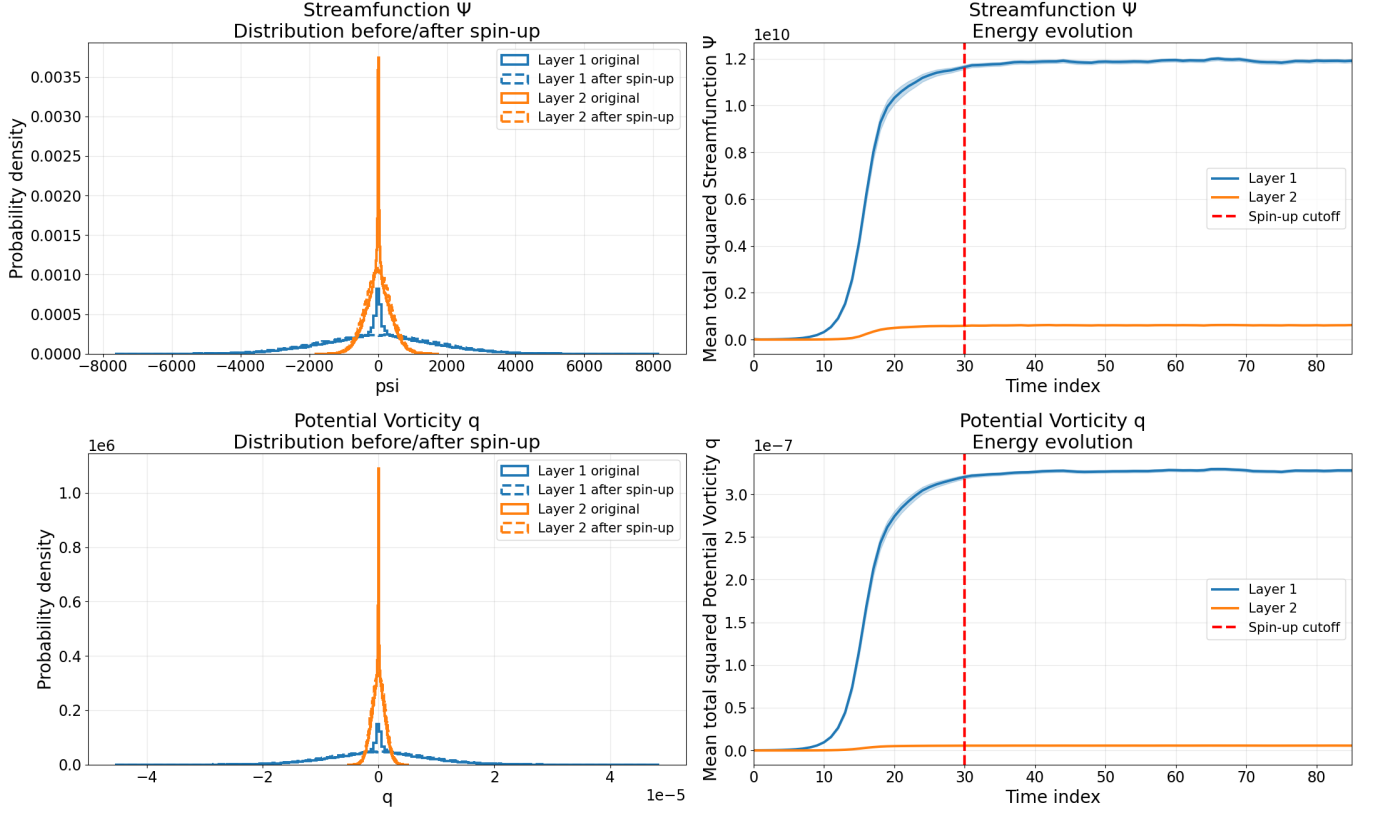


FIG. 11: Exploratory data analysis: Streamfunction ψ (top row) and Potential Vorticity q (bottom row). Left-hand plots show the probability density of each variable for layers 1 and 2, before and after discarding the first 30 time steps, which are considered the system’s spin-up period. Right-hand plots show the energy evolution given by the mean total squared value of the fields averaged across several simulations from the dataset. The red dashed line marks the spin-up cutoff, while the narrow shaded band following the energy curves indicates the 95% confidence interval.

Model	Training time [min]	Inference time [sec]	Peak Memory Use [MB]
FNO (Weighted MSE)	19.3	5.59	675.3
UNet (Physics Loss)	16.0	5.1	585.5
Weighted ChebGCN (Gradient MSE)	231	8.27	894.6
GCN (Gradient MSE)	132	4.33	791.2

TABLE VI: Computational cost for the four best performing models, all trained on ML Nodes (UiO), using an NVIDIA GeForce RTX 2080 Ti (11.5GB). Training time is given in minutes, inference time is given in seconds to generate autoregressive predictions for the whole test set (2205 samples / 4 forecast steps = 8820 forecasts in total). Peak memory is measured during inference and is given in MegaBytes.

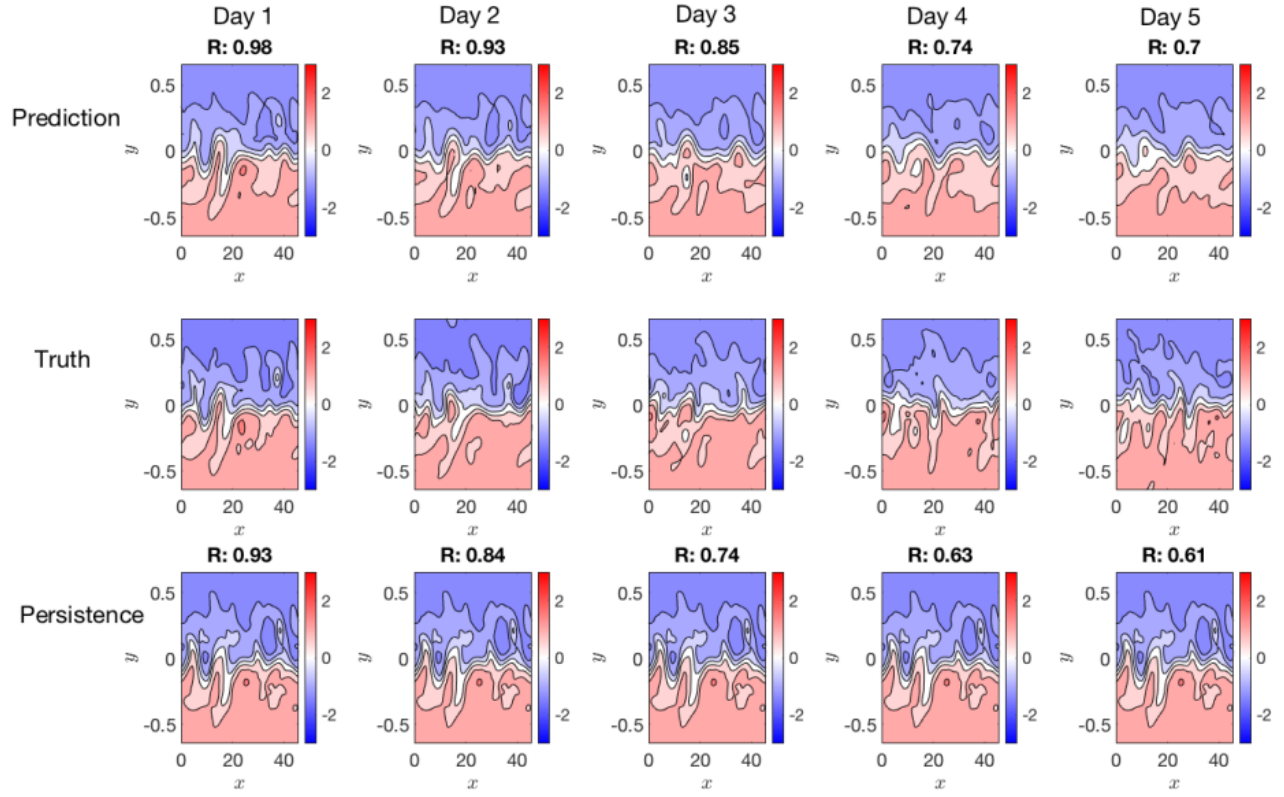


FIG. 12: Results from prior research: Future prediction of the upper ocean turbulence layer from the study of Chattopadhyay et al., Figure copied from (Chattopadhyay *et al.*, 2020).

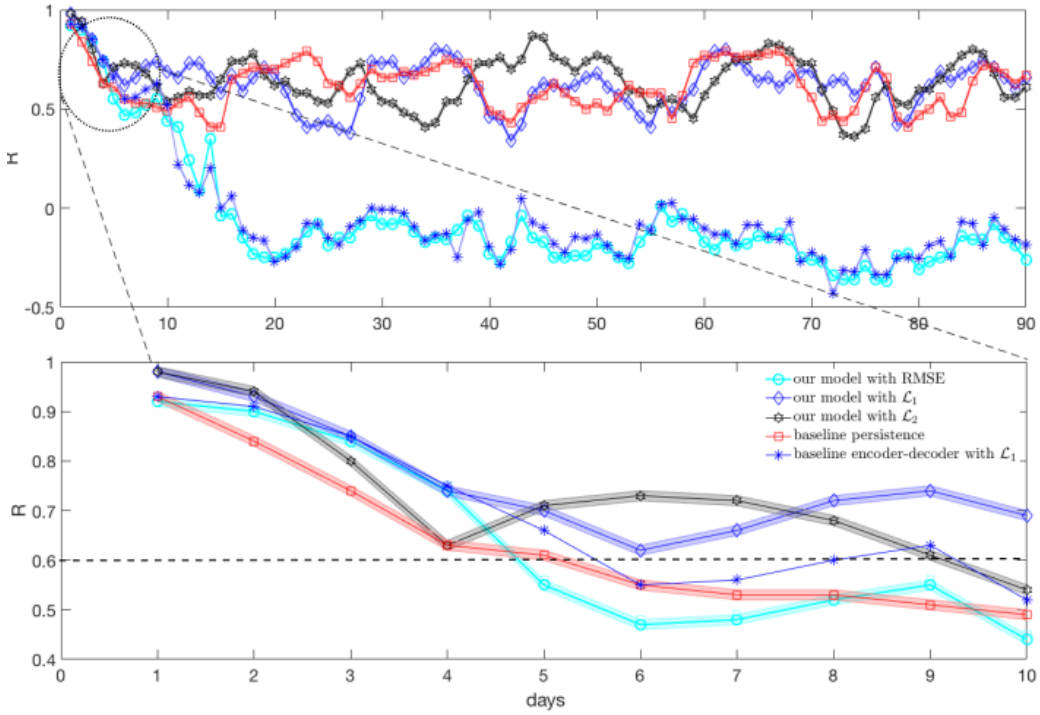


FIG. 13: Results from prior research: Evaluation of a 90-day-long prediction horizon of the model used in the study of Chattopadhyay et al. Figure copied from (Chattopadhyay *et al.*, 2020).

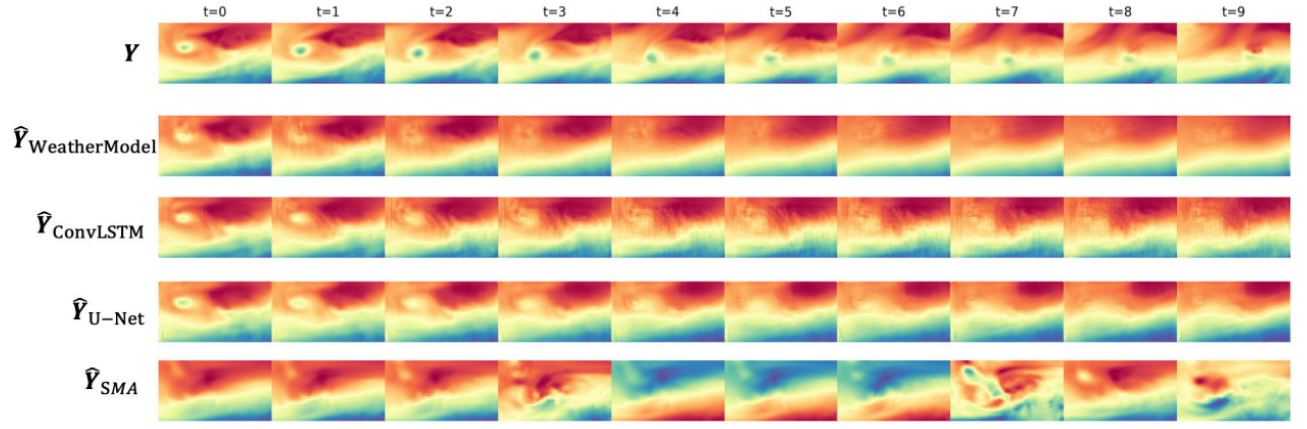


FIG. 14: Results from prior research: Weather predictions over a 72-hour prediction horizon, trained, evaluated, and tested on 1-hour temporal resolution and spatial resolution of 32×64 on a grid over the Earth. Figure copied from (Furkan Tekin *et al.*, 2024).